

Developing Full Stack Next.js Web Applications

Dr. Jose Annunziato

Table of Contents

Chapter 1 - Building Next.js User Interfaces with HTML	4
1.1 Learning Objectives	5
1.2 Setting Up the Development Environment	5
1.3 Introduction to HTML	9
1.4 Prototyping the React Kambaz User Interface with HTML	20
1.5 Committing Code to Source Control	33
1.6 Deploying Next.js Projects to the Web	36
1.7 Conclusion	36
Chapter 2 - Styling Web Pages with CSS	37
2.1 Styling React Components with CSS (Cascading Style Sheets)	37
2.2 Decorating Documents with React Icons	53
2.3 Styling Webpages with the React Bootstrap CSS Library	54
2.4 Styling Kambaz with CSS and Bootstrap	66
2.6 Delivery	79
Chapter 3 - Creating Single Page Applications with React	81
3.1 Learning Objectives	81
3.2 Introduction to JavaScript	81
3.3 JavaScript Functions	86
3.4 JavaScript Data Structures	88
3.5 Dynamic Styling	98
3.6 Parameterizing Components	100
3.7 Rendering a Data Structure	104
3.8 Debugging	105
3.8 Implementing a Data Driven Kambaz Application	106
3.9 Deliverables	113
Chapter 4 - Maintaining State in React Applications	115
4.1 Learning Objectives	115
4.2 Managing State and User Input with Forms	115
4.3 Managing Application State with Redux	124
4.4 Adding State to the Kambaz User Interface	133
4.5 Deliverables	151
Chapter 5 - Implementing RESTful Web APIs with Express.js	152
5.1 Installing and Configuring an HTTP Web Server	152
5.2 Lab Exercises	157
5.3 Implementing the Kambaz Node.js HTTP Server	181
5.4 Deploying RESTful Web Service APIs to a Public Remote Server	204
5.5 Conclusion	207
5.6 Deliverables	207
Chapter 6 - Integrating React with MongoDB	208
6.1 Working with a Local MongoDB Instance	209
6.2 Programming with a MongoDB Database	211

6.3 Integrating with MongoDB Hosted in Atlas Cloud Service.....	225
6.4 Integrating the Kambaz Web Application with a Database.....	227
6.5 Deliverables.....	246
7 References.....	247

Chapter 2 - Styling Web Pages with CSS

Web pages are formatted with **HTML**, declaring how browsers render text in various **styles**, that is, the collection of visual attributes of text such as font, size, and color. For instance, **heading tags** format their text to be much larger than their surrounding plain text. **Paragraph tags** add vertical spacing. **List tags** enumerate or bullet the line items they surround. **Table tags** format data into rows and columns. The default styling of Web pages is determined by a browser's **style sheet** written in **CSS (Cascade Style Sheets)**, a declarative language that **declares** how each **HTML** element styles the text they are applied to.

Developers use **CSS** to write custom **stylesheet documents** that override the default styling. **CSS** works by referring to different parts of the **DOM** and configuring various style attributes such as foreground and background color, font, alignment, spacing, borders, paddings, etc. This chapter demonstrates how to style and layout Web pages using CSS, a powerful declarative language.

Various patterns and best practices have evolved over time in the industry that have become popular. Some of these have been collected into commercial and open source libraries such as [Bootstrap](#), [Foundation](#), [Tailwind](#), [Material Design](#), and [Bulma](#). All these libraries define a set of **CSS rules** that can be readily applied to achieve a professional look and feel, powerful layouts, and **responsive designs**. Using a library consists of becoming familiar with the CSS rules and applying them appropriately to HTML to achieve a particular visual goal. For this course we are going to be using the **Bootstrap** CSS library, but feel free to explore and use other libraries for your **final project**.

The next section gives an opportunity to practice various **CSS** concepts and the **Bootstrap** library. After completing the practice exercises, apply the skills in the **Kambaz** section to style the Web application started in the previous chapter. Create a new branch called **a2** and do all the work described in this chapter in that branch. When done, add, commit and push the branch to GitHub. Deploy the new branch to **Vercel** and confirm it's available in a new URL, based on the branch name **a2**.

2.1 Styling React Components with CSS (Cascading Style Sheets)

This section introduces several [CSS](#), [Bootstrap](#), and [React Icons](#) exercises to practice and learn how to style HTML documents. Use the same project created the last chapter. Using **VS Code**, open the project created in the previous chapter (**kambaz-next-js**), and complete all the exercises under the **src** directory. If not already done, under the **app/labs** directory, create a new directory called **lab2**, create **page.tsx** under **app/labs/lab2** and do all work in a new **page.tsx** file. Make sure to add links in **app/labs/page.tsx** and **app/labs/TOC.tsx** to the new exercises in **app/labs/lab2/page.tsx**. Confirm you can navigate to the new exercises from the table of content.

2.1.1 Styling HTML Tags with the Style Attribute

An HTML tag's **style** attribute can configure the look and feel of the tag by changing the values of its style **properties** as shown below. The value of the **style** attribute is an object in **JSON** format (**JavaScript Object Notation**).

```
<element style={{property1: "value1", property2: "value2"}}>
  element body
</element>
```

Examples of properties **property1** and **property2** are foreground color, background color, font size, etc. The value of the properties are primitive data types such as strings or numbers, or declared variable JSON objects. To practice using the **style** attribute, copy and paste the example below into **app/labs/lab2/page.tsx**.

app/Labs/Lab2/page.tsx

```
export default function Lab2() {
  return (
    <div id="wd-lab2">
      <h2>Lab 2 - Cascading Style Sheets</h2>
      <h3>Styling with the STYLE attribute</h3>
      <p style={{ backgroundColor: "blue", color: "white" }}>
        Style attribute allows configuring look and feel right on the element. Although it's very convenient
        it is considered bad practice and you should avoid using the style attribute
      </p>
    </div>);}
```

The exercise above styles the paragraph tag with its **style** attribute, changing the background color by setting the **backgroundColor** property to **blue** and also changing the foreground color to white by setting the **color** property to **white**. There are 100s of style attributes of which we'll only cover the most common [REF].

2.1.2 Importing CSS Documents from React

Instead of changing styles within HTML, it is a **best practice** to do all styling configuration in separate CSS files and then import the files. To practice importing **CSS** files, create a brand new file called **app/labs/lab2/index.css** in the same directory of the **app/labs/lab2/page.tsx** document, and copy and paste the following content.

app/Labs/Lab2/index.css

```
p {
  background-color: green;
  color: white;
}
```

Then, remove the **style** attribute highlighted in red below and instead, import the **index.css** file highlighted in green. Confirm that the paragraph now has a green background.

app/Labs/Lab2/page.tsx

```
import "./index.css";
export default function Lab2() {
  return (
    <div id="wd-lab2">
      <h2>Lab 2 - Cascading Style Sheets</h2>
      <h3>Styling with the STYLE attribute</h3>
      <p style={{ backgroundColor: "blue", color: "white" }}>
        Style attribute allows configuring look and feel
        right on the element. Although it's very convenient
        it is considered bad practice and you should avoid
        using the style attribute</p>
      </div> );}
```

- [Home](#)
- [Lab 1](#)
- [Lab 2](#)
- [Lab 3](#)
- [Kambaz](#)

Lab 2 - Cascading Style Sheets

Styling with the STYLE attribute

Style attribute allows configuring look and feel right on the element. Although it's very convenient it is considered bad practice and you should avoid using the style attribute

- [Home](#)
- [Lab 1](#)
- [Lab 2](#)
- [Lab 3](#)
- [Kambaz](#)

Lab 2 - Cascading Style Sheets

Styling with the STYLE attribute

Style attribute allows configuring look and feel right on the element. Although it's very convenient it is considered bad practice and you should avoid using the style attribute

Figure 2.1.2.a - Styling with the STYLE attribute

Figure 2.1.2.b - Styling with the CSS file

2.1.3 Selecting HTML Content with CSS ID Selectors

The CSS rules in previous exercises styled all paragraphs by using the name of the tag **p** and then specifying the style property values. Instead of changing the look and feel of all the tags of the same name, e.g., **p**, we can refer to a specific tag by their ID, an attribute that uniquely identifies elements in a document. To practice using **ID selectors**, in **index.css**, comment out or delete the paragraph CSS rule highlighted in red as shown below, and add the two CSS rules referring to paragraphs with IDs **id-selector-1** and **id-selector-2**, highlighted in green.

app/Labs/Lab2/index.css	app/Labs/Lab2/page.tsx
<pre>/*p { background-color: green; color: white;}*/ p#wd-id-selector-1 { background-color: red; color: white; } p#wd-id-selector-2 { background-color: yellow; color: black; }</pre>	<pre>import './index.css'; export default function Lab2() { return (<div id="wd-lab2"> <h2>Lab 2 - Cascading Style Sheets</h2> <h3>Styling with the STYLE attribute</h3> ... <div id="wd-css-id-selectors"> <h3>ID selectors</h3> <p id="wd-id-selector-1"> Instead of changing the look and feel of all the elements of the same name, e.g., P, we can refer to a specific element by its ID </p> <p id="wd-id-selector-2"> Here's another paragraph using a different ID and a different look and feel </p> </div> </div>); }</pre>

Add the code above highlighted in green to **page.tsx**, and confirm the page renders as shown in the figure below.

ID selectors

Instead of changing the look and feel of all the elements of the same name, e.g., P, we can refer to a specific element by its ID

Here's another paragraph using a different ID and a different look and feel

Figure 2.1.3 - Selecting with ID CSS selectors

2.1.4 Selecting HTML Content with CSS Class Selectors

Instead of using IDs to refer to a specific tag, the **className** attribute can select a group of tags, even if they are different types of elements. To practice using **class selectors**, copy the CSS rule below into **index.css**, and the HTML at the end of **page.tsx**. The ellipses below (...) means that there's code above and/or below that is not shown for brevity. Do not include the ellipses in actual code.

app/Labs/Lab2/index.css	app/Labs/Lab2/page.tsx
<pre>... .wd-class-selector { background-color: yellow; color: blue; } ...</pre>	<pre>... <div id="wd-css-class-selectors"> <h3>Class selectors</h3> <p className="wd-class-selector"> Instead of using IDs to refer to elements, you can use an element's CLASS attribute </p> <h4 className="wd-class-selector"> This heading has same style as paragraph above </h4> </div> ...</pre>

The example above declares a selector that transforms the background and foreground color and then applies the transformation to several tags. The above example applies the style to two elements, the paragraph and the heading. Copy the code above into the stylesheet and page, and then confirm that the browser renders as shown in the figure below.

Class selectors

Instead of using IDs to refer to elements, you can use an element's CLASS attribute

This heading has same style as paragraph above

Figure 2.1.3 - Selecting with CSS class selectors

2.1.5 Selecting HTML Content Based on the Document Structure

Selectors can be combined to refer to tags in particular places in the document. A set of selectors separated by a **space** can refer to elements in a hierarchy. For instance: **.selector1.selector2 { ... }** refers to an element whose class is **.selector2** and is inside some **ancestor** whose class is **.selector1**. Alternatively, separating class selectors with the ">" character establishes a direct parent/child relationship. To practice selecting elements using a set of selectors, copy the following content in **page.tsx**. The code below does not show ellipses as previous examples, but the code is intended to be copied at the end of the file shown. Be sure to include the code within the function's return and within the wrapping **<div>**.

app/Labs/Lab2/page.tsx

```
<div id="wd-css-document-structure">
  <div className="wd-selector-1">
    <h3>Document structure selectors</h3>
    <div className="wd-selector-2">
      Selectors can be combined to refer elements in particular
      places in the document
      <p className="wd-selector-3">
        This paragraph's red background is referenced as
        <br />
        .selector-2 .selector3<br />
        meaning the descendant of some ancestor.<br />
        <span className="wd-selector-4">
          Whereas this span is a direct child of its parent
        </span><br />
        You can combine these relationships to create specific
        styles depending on the document structure
      </p>
    </div>
  </div>
</div>
```

/ this is parent element with selector
.wd-selector-1
.wd-selector-2 is a direct child of
.wd-selector-1
.wd-selector-3 is a direct child of
.wd-selector-2 and a descendant of
.wd-selector-1
.wd-selector-4 is a direct child of
.wd-selector-3 and is a descendant of
.wd-selector-2 and is a descendant of
.wd-selector-1
/

Now, in **index.css**, style tags **.wd-selector-3** and **.wd-selector-4** as shown below.

app/Labs/Lab2/index.css

```
.wd-selector-1 .wd-selector-3 {
  background-color: red;
  color: white; }
.wd-selector-2 > .wd-selector-3 > .wd-selector-4 {
  background-color: yellow;
  color: blue; }
```

/ refers to .wd-selector-3 as a descendant
of .wd-selector-1 */
/* refers to .wd-selector-4 as a direct child
of .wd-selector-3 which is a direct child
of .wd-selector-2 */*

Confirm that the page renders as shown below.

Document structure selectors

Selectors can be combined to refer elements in particular places in the document

This paragraph's red background is referenced as
.selector-2 .selector3
meaning the descendant of some ancestor.
Whereas this span is a direct child of its parent
You can combine these relationships to create specific styles depending on the
document structure

Figure 2.1.5 - Selecting HTML Content Based on the Document Structure

2.1.6 CSS Selection Rule Mechanism

CSS rules override the default styling of the tags the rules refer to. That means that there are at least two rules that apply to the same element, the default rule declared by the browser, and then the custom rule that overrides the default rule. The custom rules wins out over the default rule, but why? The **Cascading in Cascading Style Sheets** refer to the way CSS rules are applied to HTML elements based on a hierarchy of specificity and origin. This mechanism determines how styles are applied when there are multiple style rules that could affect an element. Here's how it works:

1. **Specificity:** CSS rules are applied based on their specificity. This is a measure of how precise a selector is. For example, since ID selectors only refer to a single tag, whereas class selectors can refer to many tags, the ID selector is more specific than a class selector. Since a tag's name refers to all tags with that name regardless of ID or class, a class selector is more specific than a tag name selector. More specific selectors override more general ones.
2. **Source Order:** If multiple rules have the same specificity, the last rule defined in the CSS will take precedence.
3. **Inheritance:** Some styles are inherited by child elements from their parent elements, such as font styles. However, properties like width and margin are not inherited.

2.1.7 Styling an HTML Tag's Foreground Color with CSS

Foreground colors can be configured using the CSS **color** property as follows

```
.some-css-selector { /* selects some DOM element */  
  color: blue;        /* sets color property to blue */  
}
```

Colors can be defined in various ways:

- As strings, e.g., **white**, **red**, **blue**, etc. There are thousands of declared color names. [REF]
- As hexadecimals, e.g., #ABCDEF, where each group of 2 digits, from left to right specify the amount of red, green and blue.
- As RGB, e.g., rgb(12, 34, 56), where each digit, from left to right, specifies the amount of red, green and blue.

Here are a couple of examples:

<code>.the-sun { color: rgb(255,255,0); }</code>	<code>.the-sky { color: blue; }</code>	<code>.ketchup { color: #FF0000; }</code>

To practice working with foreground colors, copy the CSS rules below into *index.css* to declare several useful color classes. Copy the HTML code below into a new file called *ForegroundColors.tsx*. Import the file into *Lab2* and confirm it renders as shown.

app/Labs/Lab2/index.css	app/Labs/Lab2/ForegroundColors.tsx
<pre>.wd-fg-color-black { color: black; } .wd-fg-color-white { color: white; } .wd-fg-color-blue { color: #7070ff; } .wd-fg-color-red { color: #ff7070; } .wd-fg-color-green { color: green; }</pre>	<pre><div id="wd-css-colors"> <h2>Colors</h2> <h3 className="wd-fg-color-blue">Foreground color</h3> <p className="wd-fg-color-red"> The text in this paragraph is red but this text is green </p> </div></pre>

Colors

Foreground color

The text in this paragraph is red but this text is green

Figure 2.1.7 - Foreground color

2.1.8 Styling an HTML Tag's Background Color with CSS

Similar to styling foreground color, the *background-color* CSS property is used to configure the background color of any tag. To practice working with background colors, copy the CSS rules shown below into *index.css* and the HTML code into a new file called *BackgroundColors.tsx*. Import the file into *Lab2* and confirm the browser renders as shown.

app/Labs/Lab2/index.css	app/Labs/Lab2/BackgroundColors.tsx
<pre>.wd-bg-color-yellow { background-color: #ffff07; } .wd-bg-color-blue { background-color: #7070ff; } .wd-bg-color-red { background-color: #ff7070; } .wd-bg-color-green { background-color: green; } .wd-bg-color-gray { background-color: lightgray; }</pre>	<pre><div id="wd-css-background-colors"> <h3 className="wd-bg-color-blue wd-fg-color-white">Background color</h3> <p className="wd-bg-color-red wd-fg-color-black"> This background of this paragraph is red but the background of this text is green and the foreground white </p> </div></pre>

Background color

This background of this paragraph is red but the background of this text is green and the foreground white

Figure 2.1.8 - Background color

2.1.9 Styling an HTML Tag's Borders with CSS

CSS has several properties that configure the look and feel of tag border around their content. Here's a sample of the properties that can be configured.

```
.some-selector {                                /* configure border with several properties */
  border-width: 10px;                          /* border's width. Can also provide per border */
  border-style: solid|dotted|dashed|double;    /* the style of the border */
  border-color: red | blue ...;               /* the color of the border */
}
```

To practice styling borders, copy the CSS code below into **index.css** and the HTML code into a new file called **Borders.tsx**. Import the new file into the **Lab2** page and confirm the browser renders as shown. Note how several classes are applied to the same tag to combine their cumulative styles to that tag.

app/Labs/Lab2/index.css	app/Labs/Lab2/Borders.tsx
<pre>.wd-border-fat { border-width: 20px 30px 20px 30px; } .wd-border-thin { border-width: 4px; } .wd-border-solid { border-style: solid; } .wd-border-dashed { border-style: dashed; } .wd-border-yellow { border-color: #ffff07; } .wd-border-red { border-color: #ff7070; } .wd-border-blue { border-color: #7070ff; }</pre>	<pre><div id="wd-css-borders"> <h2>Borders</h2> <p className="wd-border-fat wd-border-red wd-border-solid"> Solid fat red border</p> <p className="wd-border-thin wd-border-blue wd-border-dashed"> Dashed thin blue border </p> </div></pre>

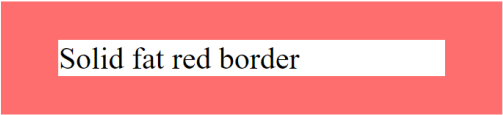


Figure 2.1.9.a - Solid fat red border



Figure 2.1.9.b - Dashed thin blue border

2.1.10 Styling an HTML Tag's Margins and Paddings with CSS

White space such as **padding** and **margin** can be styled with various CSS properties. Each side can be styled separately, e.g., top, bottom, left, right, or all the space around the content. To practice styling **padding**, copy the CSS code below into **index.css** and the HTML into **Padding.tsx**. Import and confirm the browser renders as shown.

app/Labs/Lab2/index.css	app/Labs/Lab2/Padding.tsx
<pre>.wd-padded-top-left { padding-top: 50px; padding-left: 50px; } .wd-padded-bottom-right { padding-bottom: 50px; padding-right: 50px; } .wd-padding-fat { padding: 50px; }</pre>	<pre><div id="wd-css-paddings"> <h2>Padding</h2> <div className="wd-padded-top-left wd-border-fat wd-border-red wd-border-solid wd-bg-color-yellow"> Padded top left </div> <div className="wd-padded-bottom-right wd-border-fat wd-border-blue wd-border-solid wd-bg-color-yellow"> Padded bottom right </div> <div className="wd-padding-fat wd-border-fat wd-border-yellow wd-border-solid wd-bg-color-blue wd-fg-color-white"> Padded all around </div> </div></pre>

Confirm that **Paddings.tsx** renders as shown in the figures below.



Figure 2.1.10.a - Padding top left

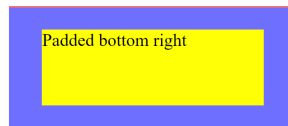


Figure 2.1.10.b - Padded bottom right



Figure 2.1.10.c - Padded all around

To practice styling **margins**, copy the CSS code below into **index.css** and the HTML into **Margins.tsx**. Import and confirm the browser renders as shown.

app/Labs/Lab2/index.css	app/Labs/Lab2/Margins.tsx
<pre>.wd-margin-bottom { margin-bottom: 50px; } .wd-margin-right-left { margin-left: 50px; margin-right: 50px; } .wd-margin-all-around { margin: 30px; }</pre>	<pre><div id="wd-css-margins"> <h2>Margins</h2> <div className="wd-margin-bottom wd-padded-top-left wd-border-fat wd-border-red wd-border-solid wd-bg-color-yellow"> Margin bottom </div> <div className="wd-margin-right-left wd-padded-bottom-right wd-border-fat wd-border-blue wd-border-solid wd-bg-color-yellow"> Margin left right </div> <div className="wd-margin-all-around wd-padding-fat wd-border-fat wd-border-yellow wd-border-solid wd-bg-color-blue wd-fg-color-white"> Margin all around </div> </div></pre>

Confirm that **Margins.tsx** renders as shown in the figures below.

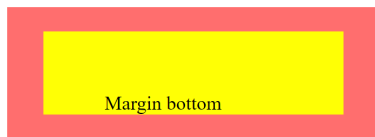


Figure 2.1.10.d - Margin bottom

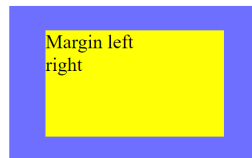


Figure 2.1.10.e - Margin left and right



Figure 2.1.10.f - Margin all around

2.1.11 Styling an HTML Tag's Corners with CSS

Tag border corners can be styled to be rounded to soften their look and feel. All corners can be styled together or individual corners can be styled separately. How rounded a corner is can be configured by setting the corner's border radius. To practice rounding some corners, copy the CSS and HTML below into **index.css** and **Corners.tsx**. Import and confirm the browser renders as shown.

app/Labs/Lab2/index.css	app/Labs/Lab2/Corners.tsx
<pre>.wd-rounded-corners-top { border-top-left-radius: 40px; border-top-right-radius: 40px; } .wd-rounded-corners-bottom { border-bottom-left-radius: 40px;</pre>	<pre><div id="wd-css-borders"> <h3>Rounded corners</h3> <p className="wd-rounded-corners-top wd-border-thin wd-border-blue wd-border-solid wd-padding-fat"> Rounded corners on the top </p> <p className="wd-rounded-corners-bottom</pre>

<pre>border-bottom-right-radius: 40px; } .wd-rounded-corners-all-around { border-radius: 50px; } .wd-rounded-corners-inline { border-radius: 30px 0px 20px 50px; }</pre>	<pre>wd-border-thin wd-border-blue wd-border-solid wd-padding-fat"> Rounded corners at the bottom </p> <p className="wd-rounded-corners-all-around wd-border-thin wd-border-blue wd-border-solid wd-padding-fat"> Rounded corners all around </p> <p className="wd-rounded-corners-inline wd-border-thin wd-border-blue wd-border-solid wd-padding-fat"> Different rounded corners </p> </div></pre>
--	---

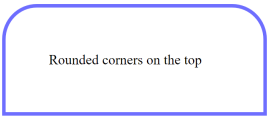


Figure 2.1.11.a - Rounded corners on the top

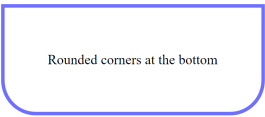


Figure 2.1.11.b - Rounded corners at the bottom

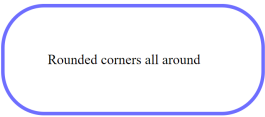


Figure 2.1.11.c - Rounded corners all around



Figure 2.1.11.d - Different rounded corners

2.1.12 Styling an HTML Tag's Dimensions with CSS

An element's dimensions can be styled with the *width* and *height* properties. To practice setting an element's dimensions, copy the CSS and HTML below into *index.css* and *Dimensions.tsx*. Import and confirm the browser renders as shown.

app/Labs/Lab2/index.css	app/Labs/Lab2/Dimensions.tsx
<pre>.wd-dimension-portrait { width: 75px; height: 100px; } .wd-dimension-landscape { width: 100px; height: 75px; } .wd-dimension-square { width: 75px; height: 75px; }</pre>	<pre><div id="wd-css-dimensions"> <h2>Dimension</h2> <div> <div className="wd-dimension-portrait wd-bg-color-yellow"> Portrait </div> <div className="wd-dimension-landscape wd-bg-color-blue wd-fg-color-white"> Landscape </div> <div className="wd-dimension-square wd-bg-color-red"> Square</div> </div> </div></pre>

By default, block elements such as `<div>`, *headings*, and *paragraphs* span the full width of their parent container—often the entire browser window—as shown on the left below. You can override these dimensions using the CSS width and height properties, as demonstrated on the right. Even when the width is reduced to make the elements skinnier, increasing their height (or adding margins/padding) still pushes subsequent block elements further down the page, since block elements naturally stack vertically and occupy space based on their total height.

Dimension



Figure 2.1.12.a - Styling an HTML Tag's Dimensions with CSS

Dimension



Figure 2.1.12.b - Styling an HTML Tag's Dimensions with CSS

2.1.13 Styling an HTML Tag's Relative Position with CSS

Browsers automatically determine the default position of all content displayed on the screen. You can override these default positions using the CSS *position* property. This property supports several values, with the most common ones being *relative*, *absolute*, *static*, and *fixed*. Setting the position property to *relative* allows you to shift an element from its default position without affecting the layout of surrounding elements. You can then use the *top*, *bottom*, *left*, and *right* properties to specify the direction and distance of the shift. To practice positioning elements relatively, copy the CSS and HTML code provided below into *index.css* and *Positions.tsx*, respectively. Verify that your browser renders the result as shown in Figure 2.1.13. Note that the background colors referenced in this exercise were defined in a previous exercise.

app/Labs/Lab2/index.css	app/Labs/Lab2/Positions.tsx
<pre>.wd-pos-relative-nudge-up-right { position: relative; bottom: 30px; left: 30px; } .wd-pos-relative-nudge-down-right { position: relative; top: 20px; left: 20px; } .wd-pos-relative { position: relative; }</pre>	<pre><div id="wd-css-position-relative"> <h2>Relative</h2> <div className="wd-bg-color-gray"> <div className="wd-bg-color-yellow wd-dimension-portrait"> <div className="wd-pos-relative-nudge-down-right"> Portrait</div></div> <div className="wd-pos-relative-nudge-up-right wd-bg-color-blue wd-fg-color-white wd-dimension-landscape"> Landscape</div> <div className="wd-bg-color-red wd-dimension-square"> Square</div> </div> </div></pre>

2.1.14 Styling a Tag's Absolute Position with CSS

The CSS *position* property set to *absolute* positions an element relative to its nearest ancestor with a position value of *relative*, *absolute*, or *fixed*. If no such ancestor exists, the element is positioned relative to the *body* tag, effectively aligning with the browser's *viewport*. To practice using *absolute* positioning, copy the CSS and HTML code provided below into *index.css* and *Positions.tsx*, respectively, and verify the browser renders the result as shown in the figure below. Note that several *br* elements are included at the end of the example to create space for the next exercise.

app/Labs/Lab2/index.css	app/Labs/Lab2/Positions.tsx
<pre> .wd-pos-absolute-10-10 { position: absolute; top: 10px; left: 10px; } .wd-pos-absolute-50-50 { position: absolute; top: 50px; left: 50px; } .wd-pos-absolute-120-20 { position: absolute; top: 20px; left: 120px; } </pre>	<pre> <div id="wd-css-position-absolute"> <h2>Absolute position</h2> <div className="wd-pos-relative"> <div className="wd-pos-absolute-10-10 wd-bg-color-yellow wd-dimension-portrait"> Portrait</div> <div className="wd-pos-absolute-50-50 wd-bg-color-blue wd-fg-color-white wd-dimension-landscape"> Landscape</div> <div className="wd-pos-absolute-120-20 wd-bg-color-red wd-dimension-square"> Square</div> </div>

 </div> </pre>

The term "**absolute**" can be misleading, as it positions the element relative to a parent or the **body** tag rather than an absolute point on the screen. Additionally, an element with **absolute** positioning scrolls with the rest of the page content. In contrast, when precise positioning is needed regardless of other elements or scrolling, the **fixed** position property, covered in the next section, anchors an element relative to the browser's viewport.

Relative

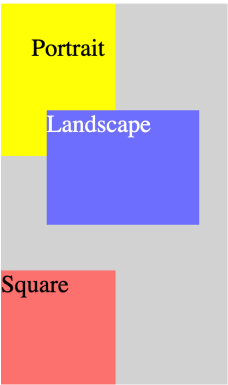


Figure 2.1.13 - Relative position

Absolute position

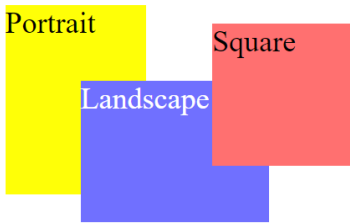


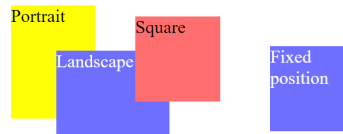
Figure 2.1.4 - Absolute position

2.1.15 Styling an HTML Tag's Fixed Position with CSS

Setting the CSS **position** property to **fixed** positions an element relative to the browser's **viewport**. This ensures that the element remains in place, even when the page content is scrolled. To practice using **fixed** positioning, copy the CSS and HTML code provided below into **index.css** and **Positions.tsx**, respectively, and verify that the browser renders the result as shown in the figure below. Note that the appearance may vary slightly depending on your screen size and scroll position.

app/Labs/Lab2/index.css	app/Labs/Lab2/Positions.tsx
<pre> .wd-pos-fixed { position: fixed; right: 0px; bottom: 50%; } </pre>	<pre> <div id="wd-css-position-fixed"> <h2>Fixed position</h2> Checkout the blue square that says "Fixed position" stuck all the way on the right and half way down the page. It doesn't scroll with the rest of the page. Its position is "Fixed". <div className="wd-pos-fixed wd-dimension-square wd-bg-color-blue wd-fg-color-white"> Fixed position </div> </div> </pre>

Absolute position



Fixed position

Checkout the blue square that says "Fixed position" stuck all the way on the right and half way down the page. It doesn't scroll with the rest of the page. Its position is "Fixed".

Figure 2.1.15 - Fixed position

2.1.16 Styling an HTML Tag's Z Index with CSS

When rendering HTML content, the browser automatically calculates the positions and dimensions of elements, assigning each a dedicated rectangular space on the screen to prevent overlap. However, using the CSS **position** property to override default positioning can lead to elements overlapping. By default, elements are rendered in the order they appear in the HTML, meaning later-declared elements appear above earlier ones when they overlap. The **z-index** CSS property allows you to control this stacking order: elements with a higher **z-index** value are displayed above those with a lower value. The default value of **z-index** is **auto** where elements are rendered in the order they are declared. To practice setting an element's **z-index**, copy the CSS and HTML code provided below into **index.css** and **Zindex.tsx**, respectively, and confirm that the browser renders the result as shown in the figure below.

<i>app/Labs/Lab2/index.css</i>	<i>app/Labs/Lab2/Zindex.tsx</i>
<pre>.wd-zindex-bring-to-front { z-index: 10; }</pre>	<pre><div id="wd-z-index"> <h2>Z index</h2> <div className="wd-pos-relative"> <div className="wd-pos-absolute-10-10 wd-bg-color-yellow wd-dimension-portrait"> Portrait </div> <div className="wd-zindex-bring-to-front wd-pos-absolute-50-50 wd-dimension-landscape wd-bg-color-blue wd-fg-color-white"> Landscape </div> <div className="wd-pos-absolute-120-20 wd-bg-color-red wd-dimension-square"> Square </div> </div>

 </div></pre>

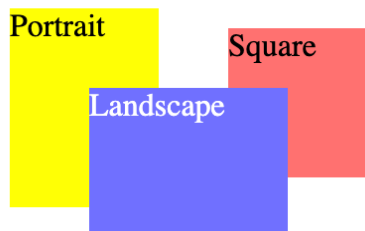


Figure 2.1.16 - Z-Index

2.1.17 Floating Images and Content with CSS

The CSS **float** property aligns elements to the left or right edges of the browser window while allowing adjacent content to wrap around the floated element. For instance consider the screen shot shown in Figure 2.1.17.a where long paragraphs

flow around images aligned to the right and to the left. To accomplish this layout, images can be inserted in between several long portions of text. Normally images would flow inline with the rest of the adjacent text, but if the images are **float**ed, they align with the edges of the browser and the text would then flow around the images. The text in the screen shot is **lorem ipsum**, a popular placeholder text often used when designing Web pages and other printed media. It can be found in <https://www.lipsum.com>.

app/Labs/Lab2/index.css	app/Labs/Lab2/Float.tsx
<pre>.wd-float-left { float: left; height: 100px; } .wd-float-right { float: right; height: 100px; } .wd-float-done { clear: both; }</pre>	<pre><div id="wd-float-divs"> <h2>Float</h2> <div> Lorem ipsum, dolor sit amet consectetur adipisicing elit. Eius hic ... Lorem ipsum, dolor sit amet consectetur adipisicing elit. Eius hic ... Lorem ipsum, dolor sit amet consectetur adipisicing elit. Eius hic ... Lorem ipsum, dolor sit amet consectetur adipisicing elit. Eius hic ... Lorem ipsum, dolor sit amet consectetur adipisicing elit. Eius hic ... Lorem ipsum, dolor sit amet consectetur adipisicing elit. Eius hic ... Lorem ipsum, dolor sit amet consectetur adipisicing elit. Eius hic ... Lorem ipsum, dolor sit amet consectetur adipisicing elit. Eius hic ... <div className="wd-float-done"></div> </div> </div></pre>

Use the CSS and HTML code snippets above to practice using the **float** property to layout images as shown in the figure. Copy longer portions of the **lorem ipsum** text from the reference, or feel free to use your own text. Implement the example in a new **Float.tsx** file, import it and confirm it renders as shown. By default, HTML alone cannot arrange content horizontally, except through methods like **table** columns. The CSS **float** property can be applied to **div** elements to create horizontal layouts, as demonstrated in Figure 2.1.17.b. To practice this horizontal arrangement, copy the CSS and HTML code provided below into **Float.tsx**, import it, and confirm that the browser renders the layout as shown in the figure below.

app/Labs/Lab2/index.css	app/Labs/Lab2/Float.tsx
<pre>.wd-float-left { float: left; height: 100px; } .wd-float-right { float: right; height: 100px; } .wd-float-done { clear: both; }</pre>	<pre><div id="wd-float-divs"> <h2>Float</h2> <div> <div className="wd-float-left wd-dimension-portrait wd-bg-color-yellow"> Yellow </div> <div className="wd-float-left wd-dimension-portrait wd-bg-color-blue wd-fg-color-white"> Blue </div> <div className="wd-float-left wd-dimension-portrait wd-bg-color-red"> Red </div> <div className="wd-float-done"></div> </div> </div></pre>



Figure 2.1.17.b - Floating Images and Content with CSS

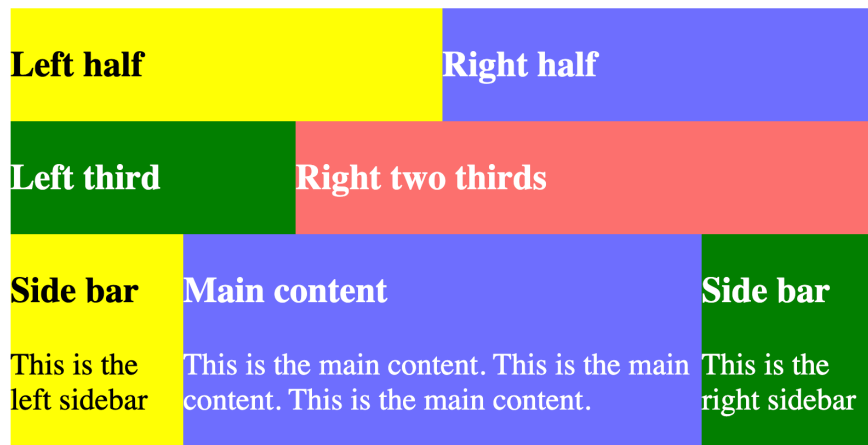


Figure 2.1.18 - Laying Out Content in a Grid Using CSS

2.1.19 Laying Out Content with CSS Flex

Flexbox Layout (Flexible Box, or just Flex) provides a simpler way to layout and distribute content in an HTML document. It all starts with creating a container element that configures the layout and behavior of its child elements. To illustrate some of the features of flex, create a container with the **display** property set to **flex** as shown below.

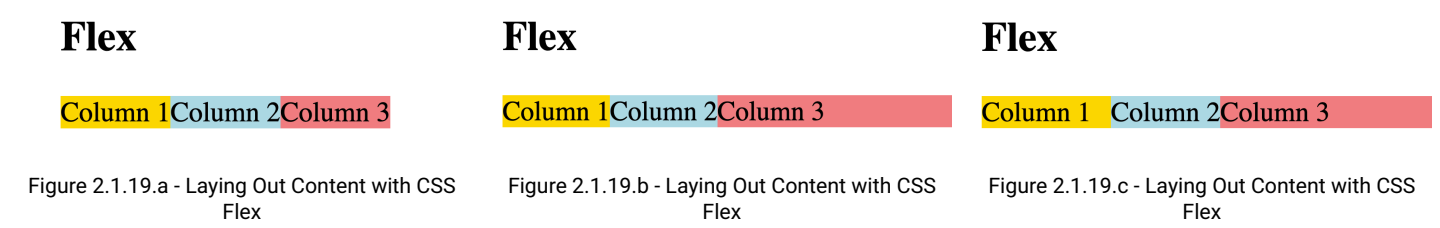
<code>app/Labs/Lab2/Flex.tsx</code>	<code>app/Labs/Lab2/index.css</code>
<pre><div id="wd-css-flex"> <h2>Flex</h2> <div className="wd-flex-row-container"> <div className="wd-bg-color-yellow">Column 1</div> <div className="wd-bg-color-blue">Column 2</div> <div className="wd-bg-color-red">Column 3</div> </div> </div></pre>	<pre>.wd-flex-row-container { display: flex; flex-direction: row; }</pre>

Note how DIV elements inside the container render horizontally as a row of element instead of stacking the elements vertically. Flex simplifies laying out content horizontally. Flex child elements can also be configured to grow and expand to fill into empty spaces. The styling below illustrates how the last column can expand into the empty space to its right.

<code>app/Labs/Lab2/Flex.tsx</code>	<code>app/Labs/Lab2/index.css</code>
<pre><div id="wd-css-flex"> <h2>Flex</h2> <div className="wd-flex-row-container"> <div className="wd-bg-color-yellow"> Column 1</div> <div className="wd-bg-color-blue"> Column 2</div> <div className="wd-bg-color-red wd-flex-grow-1"> Column 3</div> </div> </div></pre>	<pre>.wd-flex-row-container { display: flex; flex-direction: row; } .wd-flex-grow-1 { flex-grow: 1; }</pre>

The rest of the flex child elements can be configured independently to have specific widths to fit whatever content is needed as shown below.

app/Labs/Lab2/Flex.tsx	app/Labs/Lab2/index.css
<pre> <div id="wd-css-flex"> <h2>Flex</h2> <div className="wd-flex-row-container"> <div className="wd-bg-color-yellow wd-width-75px"> Column 1</div> <div className="wd-bg-color-blue"> Column 2</div> <div className="wd-bg-color-red wd-flex-grow-1"> Column 3</div> </div> </div> </pre>	<pre> .wd-flex-row-container { display: flex; flex-direction: row; } .wd-flex-grow-1 { flex-grow: 1; } .wd-width-75px { width: 75px; } </pre>



2.1.20 Media Queries

Media queries are a CSS feature that allow styles to be applied conditionally based on characteristics of the device or viewport, such as screen width, height, orientation, or resolution. They are essential for creating responsive web designs that adapt seamlessly to different devices, like desktops, tablets, and smartphones.

The example below demonstrates applying simple background and foreground colors based on various screen sizes. The default style, outside of any *@media at rule*, defines the default background and foreground colors as white text on a green background. Subsequent *at rules* redefine the style that only take effect at various screen size rages, e.g., black on white for screen widths between 750 and 1000 pixels.

app/Labs/Lab2/MediaQueriesDemo.css
<pre> /* Default styles */ .wd-media-queries-demo { background-color: green; color: white; } /* Style between 750px and 1000px */ @media (min-width: 750px) and (max-width: 1000px) { .wd-media-queries-demo { background-color: yellow; color: black; } } /* Style between 1000px and 1250px */ @media (min-width: 1000px) and (max-width: 1250px) { .wd-media-queries-demo { background-color: blue; color: white; } } /* Style for widths greater than 1250px */ @media (min-width: 1250px) { .wd-media-queries-demo { background-color: red; color: white; } } </pre>

The component below applies the ***wd-media-queries-demo*** class rendering the content in various combinations of background and foreground colors as the browser window is resized.

app/Labs/Lab2/MediaQueriesDemo.tsx

```
import "../MediaQueriesDemo.css";

export default function MediaQueriesDemo() {
  return (
    <div className="wd-media-queries-demo">
      <h1>Media Query Demo</h1>
      <p>
        This demo uses CSS media queries to change colors based on screen width:
      </p>
      <ul>
        <li>Default is White text on Green background</li>
        <li>750px to 1000px: Black text on Yellow background</li>
        <li>1000px to 1250px: White text on Blue background</li>
        <li>Above 1250px: White text on Red background</li>
      </ul>
    </div>
  );
}
```

Import ***MediaQueriesDemo*** into the labs page, resize the browser window, and confirm the content renders as shown below for different screen widths.

Media Query Demo

This demo uses CSS media queries to change colors based on screen width:

- Default is White text on Green background
- 750px to 1000px: Black text on Yellow background
- 1000px to 1250px: White text on Blue background
- Above 1250px: White text on Red background

Figure 2.1.20a - Default Style

Media Query Demo

This demo uses CSS media queries to change colors based on screen width:

- Default is White text on Green background
- 750px to 1000px: Black text on Yellow background
- 1000px to 1250px: White text on Blue background
- Above 1250px: White text on Red background

Figure 2.1.20b - Styles between 750px and 1000px

Media Query Demo

This demo uses CSS media queries to change colors based on screen width:

- Default is White text on Green background
- 750px to 1000px: Black text on Yellow background
- 1000px to 1250px: White text on Blue background
- Above 1250px: White text on Red background

Figure 2.1.20c - Styles between 1000px and 1250px

Media Query Demo

This demo uses CSS media queries to change colors based on screen width:

- Default is White text on Green background
- 750px to 1000px: Black text on Yellow background
- 1000px to 1250px: White text on Blue background
- Above 1250px: White text on Red background

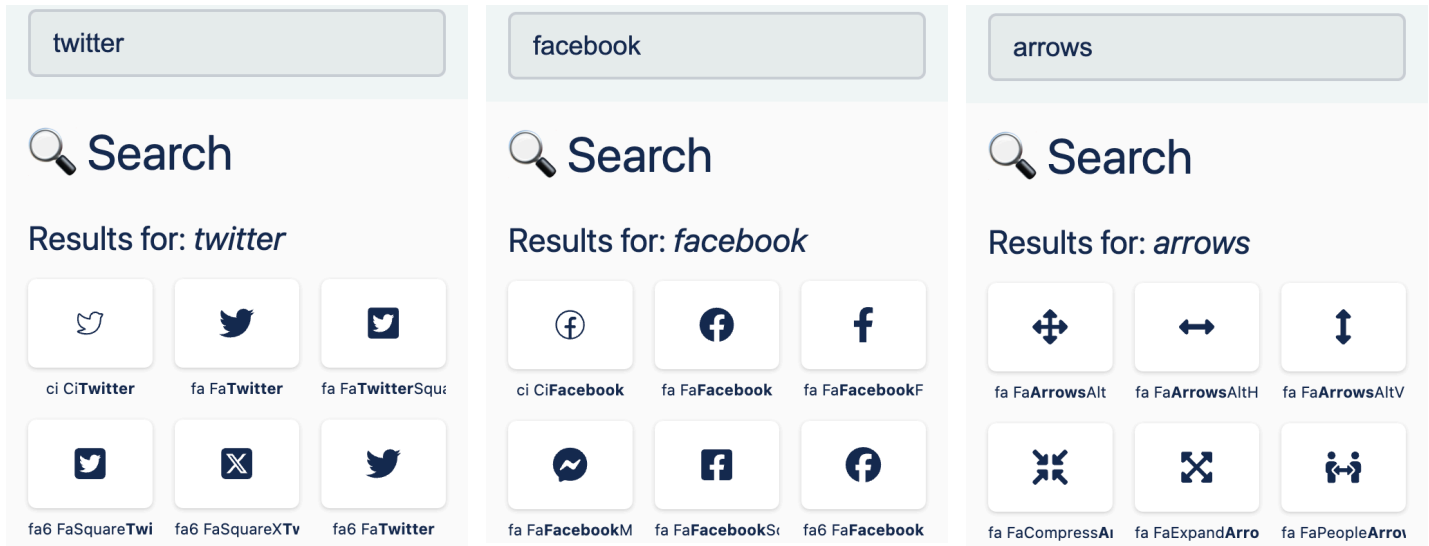
Figure 2.1.20d - Styles between 1250px and above

2.2 Decorating Documents with React Icons

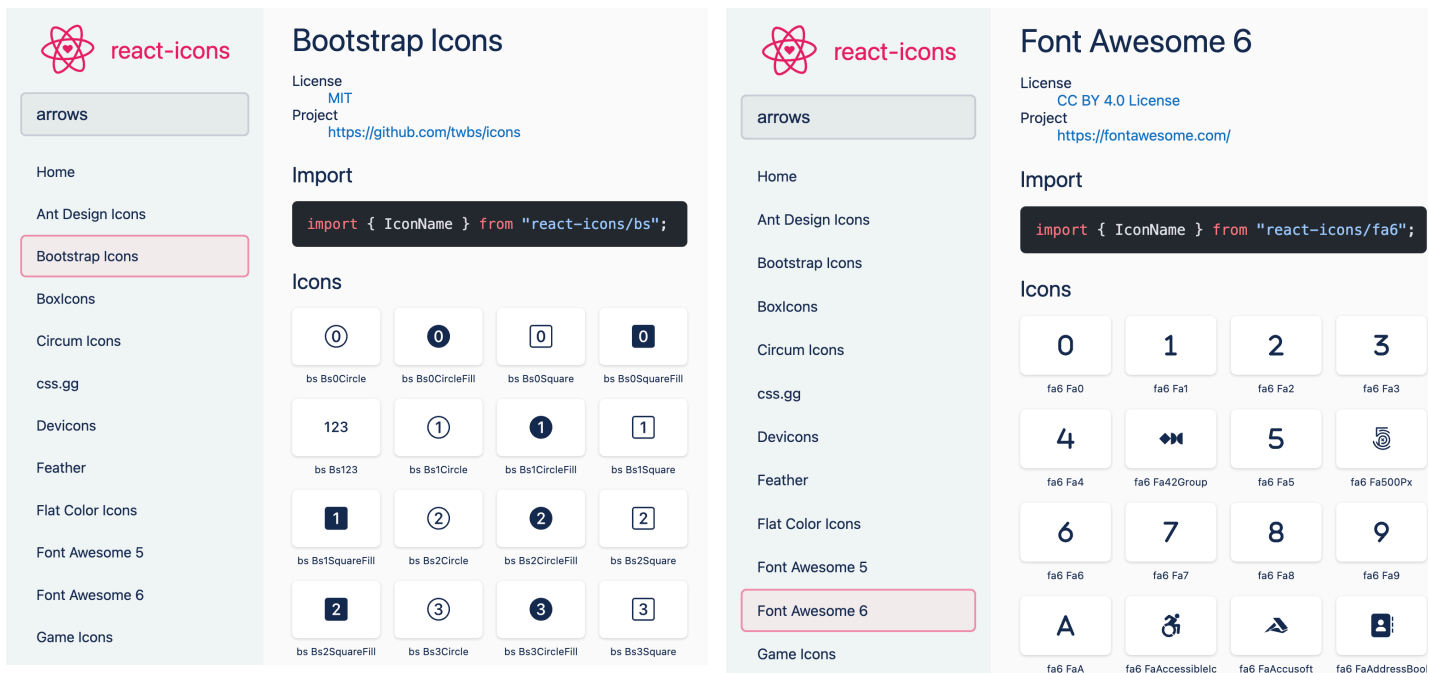
[React icons](#) is a CSS library that aggregates icons from various sources including ***Fontawesome***, ***Bootstrap***, and ***Heroicons***. Install React icons from the root of your project as shown below.

```
npm install react-icons
```

Head over to <https://react-icons.github.io/react-icons> to search for icons of interest. Search for icons by typing a topic in the search box. Here are a few example searches found from several icon sources.



Navigate through the icons by their vendors as shown below, e.g, **Bootstrap** and **Fontawesome**.



The icons are made available as **React** components that can be imported and used as tags within HTML code. To practice using **React Icons**, create the **ReactIconsSampler** component as shown below. Import the component at the end of **Lab2** page and confirm it renders as shown below on the right.

app/Labs/Lab2/ReactIconsSampler.tsx

```
import { FaCalendar, FaEnvelopeOpenText, FaRegClock } from "react-icons/fa";
import { AiOutlineDashboard } from "react-icons/ai";
import { FaBookBible } from "react-icons/fa6";
import { VscAccount } from "react-icons/vsc";
export default function ReactIconsSampler() {
  return (
    <div id="wd-react-icons-sampler" className="mb-4">
      <h3>React Icons Sampler</h3>
    </div>
  );
}
```

```

<div className="d-flex">
  <VscAccount className="fs-3 text" />
  <AiOutlineDashboard className="fs-3 text" />
  <FaBookBible className="fs-3 text" />
  <FaCalendar className="fs-3 text" />
  <FaEnvelopeOpenText className="fs-3 text" />
  <FaRegClock className="fs-3 text" />
</div>
</div>
);}

```



Figure 2.2 - Decorating Documents with React Icons

2.3 Styling Webpages with the Tailwind CSS Library

Tailwind CSS is a utility-first CSS framework that enables developers to rapidly build custom, modern user interfaces directly in their HTML by applying pre-defined utility classes instead of writing custom CSS. Tailwind CSS was preconfigured when the Next.js project was created, but we disabled it in **app/layout.tsx** so we could discuss plain HTML without adding the complexity of CSS styling.

app/layout.tsx

```

import type { Metadata } from "next";
import { Geist, Geist_Mono } from "next/font/google";
// import "./globals.css";
...

```

To explore using tailwind, let's create a CSS file where we can import the tailwind library. This will allow us to target specific pages and components that explicitly import the CSS file.

app/labs/Lab2/tailwind/index.css

```
@import "tailwindcss";
```

Create a new page where we can practice with some of the more common tailwind features.

app/labs/Lab2/tailwind/page.tsx

```

import "./index.css";

export default function TailwindLab() {
  return (
    <div className="p-8">
      <h1 className="text-4xl font-bold mb-8">Tailwind CSS</h1>
    </div>
  );
}

```

2.3.1 Spacing

The defining feature of Tailwind CSS is its **utility-first** approach, which provides low-level atomic utility classes for styling elements directly in HTML, enabling rapid custom design without writing traditional custom CSS. The [padding](#) and [margin](#)

utility classes allow defining spacing of between and around content. The component below illustrates using **margin** classes to configure spacing between content and **padding** to configure spacing surrounding the content.

app/Labs/Lab2/tailwind/TailwindSpacing.tsx

```
export default function TailwindSpacing() {
  return (
    <div>
      <h2 className="text-3xl">Margin</h2>
      <div className="bg-blue-200 mb-4 p-4">
        This div has a bottom margin of 4.
      </div>
      <div className="bg-blue-200 ms-4 me-8 p-4">
        This div has a start margin of 4 and an end margin of 8.
      </div>
      <h2 className="text-3xl mt-8">Padding</h2>
      <div className="bg-green-200 ps-2 pt-4 pb-8 mb-4">
        This div has starting padding of 2, top padding of 4, and bottom padding of 8.
      </div>
      <div className="bg-green-200 p-6">
        This div has padding all around of 6.
      </div>
    </div>
  );
}
```

Classes beginning with **m** configure margin and classes beginning with **p** configure padding. A number can be appended at to **m-** or **p-** to configure the amount of margin or padding, e.g., **m-1**, **m-2**, through **m-8**, configure increasing amounts of margin where as **p-1** through **p-8** configures increasing amounts of padding. Import the component above into the **TailwindLab** page and confirm it renders as shown below.

Margin

This div has a bottom margin of 4.

This div has a start margin of 4 and an end margin of 8.

Figure 2.3.1a - Margin Spacing

Padding

This div has starting padding of 2, top padding of 4, and bottom padding of 8.

This div has padding all around of 6.

Figure 2.3.1b - Padding Spacing

2.3.2 Typography

Tailwind CSS provides a comprehensive set of typography utility classes that allow developers to quickly control text styling—such as font size, weight, color, alignment, and more—directly in their markup without writing custom CSS.

The following React component demonstrates some common Tailwind typography classes by showcasing a range of [font sizes \(from text-sm to text-3xl\)](#) and [font weights \(from font-thin to font-black\)](#).

app/Labs/Lab2/tailwind/TailwindTypography.tsx

```
export default function TailwindTypography() {
  return (
    <div>
      <h2 className="text-3xl">Font Size</h2>
      <p className="text-sm">This is small text.</p>
      <p className="text-base">This is base text.</p>
      <p className="text-lg">This is large text.</p>
      <p className="text-xl">This is extra large text.</p>
      <p className="text-2xl">This is 2x extra large text.</p>
      <p className="text-3xl">This is 3x extra large text.</p>
    </div>
  );
}
```

```

<h2 className="text-3xl font-bold mt-4">Font Weight</h2>
<p className="font-thin">This is thin font weight.</p>
<p className="font-light">This is light font weight.</p>
<p className="font-normal">This is normal font weight.</p>
<p className="font-medium">This is medium font weight.</p>
<p className="font-semibold">This is semi-bold font weight.</p>
<p className="font-bold">This is bold font weight.</p>
<p className="font-extrabold">This is extra-bold font weight.</p>
<p className="font-black">This is black font weight.</p>
</div>
);
}

```

Import the component into the **TailwindLab** and confirm it renders as shown below.

Font Size

This is small text.

This is base text.

This is large text.

This is extra large text.

This is 2x extra large text.

This is 3x extra large text.

Figure 2.3.2a - Font Size

Font Weight

This is thin font weight.

This is light font weight.

This is normal font weight.

This is medium font weight.

This is semi-bold font weight.

This is bold font weight.

This is extra-bold font weight.

This is black font weight.

Figure 2.3.2b - Font Weight

2.3.3 Background Colors

Tailwind CSS offers a powerful and customizable palette of [background color utility classes](#) that allow developers to quickly apply solid colors, gradients, or even arbitrary values to elements directly in their markup.

The following React component demonstrates some common Tailwind background color classes by applying vibrant shades like **bg-red-500**, **bg-green-500**, **bg-blue-500**, and **bg-yellow-500** to div elements, with contrasting text colors for readability.

app/Labs/Lab2/tailwind/TailwindBackgroundColors.tsx

```

export default function TailwindBackgroundColors() {
  return (
    <div>
      <h2 className="text-3xl font-bold mb-4">Background Colors</h2>
      <div className="bg-red-500 text-white p-4 mb-4">
        This div has a red background.
      </div>
      <div className="bg-green-500 text-white p-4 mb-4">
        This div has a green background.
      </div>
      <div className="bg-blue-500 text-white p-4 mb-4">
        This div has a blue background.
      </div>
      <div className="bg-yellow-500 text-black p-4 mb-4">
        This div has a yellow background.
      </div>
    </div>
  );
}

```

```
);  
}
```

Import the component above into the **TailwindLab** page and confirm it renders as shown below.

Background Colors

This div has a red background.

This div has a green background.

This div has a blue background.

This div has a yellow background.

Figure 2.3.3 - Background Colors

2.3.4 Responsive Design

Tailwind CSS excels at [responsive design through its mobile-first breakpoints](#), allowing developers to apply utility classes prefixed with screen sizes (like sm:, md:, lg:) to adapt layouts, spacing, and styling seamlessly across devices.

The following React component demonstrates Tailwind's responsive features by creating a card that stacks vertically on smaller screens and switches to a flexible side-by-side layout with adjusted widths on medium screens and above (using classes like md:flex, md:max-w-2xl, and md:w-48).

[The code below references an image of the React logo.](#) Download the image and save it to **public/images**.

app/Labs/Lab2/tailwind/TailwindResponsiveDesign.tsx

```
export default function TailwindResponsiveDesign() {  
  return (  
    <div className="mx-auto max-w-md overflow-hidden rounded-xl bg-white shadow-md md:max-w-2xl">  
      <div className="md:flex">  
        <div className="md:shrink-0">  
            
        </div>  
        <div className="p-8">  
          <div className="text-sm font-semibold tracking-wide text-indigo-500 uppercase">  
            Professional Courses  
          </div>  
          <a href="#" className="mt-1 block text-lg leading-tight font-medium text-black hover:underline" >  
            Rocket Propulsion Fundamentals  
          </a>  
          <p className="mt-2 text-gray-500">  
            This course provides an in-depth study of the fundamentals of rocket  
            propulsion, covering topics such as propulsion theory, engine types,  
            fuel chemistry, and the practical applications of rocket technology.  
            Designed for students with a strong background in physics and  
            engineering, the course includes both theoretical instruction and  
            hands-on laboratory work  
          </p>  
        </div>  
      </div>  
    </div>  
  );  
}
```

```

    </div>
  <br />
</div>
);
}

```

Import the component above into the **TailwindLab** page and confirm it renders as shown below.



Figure 2.3.4 - Tailwind Responsive Design
Wide Screen

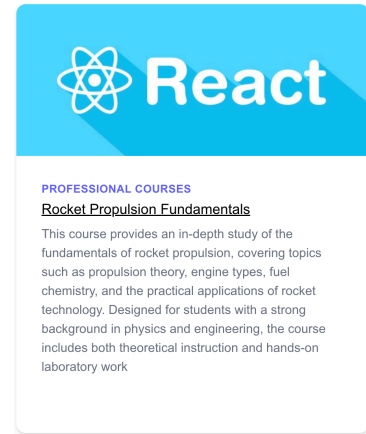


Figure 2.3.4 - Tailwind Responsive Design Narrow Screen

2.3.5 Filters

Tailwind CSS includes a robust set of [filter utility classes](#) that enable developers to easily apply visual effects like blur, brightness, contrast, grayscale, sepia, and more directly to elements such as images.

The following React component illustrates Tailwind's blur filters by displaying the same image of Angel Falls four times side-by-side, applying progressively stronger blur effects with classes ***blur-none***, ***blur-sm***, ***blur-lg***, and ***blur-2xl***.

app/Labs/Lab2/tailwind/TailwindFilters.tsx

```

export default function TailwindFilters() {
  return (
    <div>
      <div>
        <h3>Blurs</h3>
        <div className="flex">
          
          
          
          
        </div>
      </div>
    </div>
  );
}

```

Import the component above into the **TailwindLab** page and confirm it renders as shown below.



Figure 2.3.5 - Blur filters

2.3.6 CSS Grid Layout

Tailwind CSS provides powerful and intuitive grid utility classes based on CSS Grid, allowing developers to easily create complex, responsive layouts by defining columns, rows, gaps, and item placement directly in markup.

The following React component demonstrates a basic Tailwind grid by using grid ***grid-cols-4 gap-4*** to create a 4-column layout where nine numbered items automatically flow and wrap into multiple rows.

app/Labs/Lab2/tailwind/TailwindGrids.tsx

```
export default function TailwindGrids() {
  return (
    <div>
      <h2>Tailwind Grids</h2>
      <div>
        <h3 className="mt-6 text-3xl font-bold">4 Columns Grid</h3>
        <div className="grid grid-cols-4 gap-4">
          <div className="text-center bg-blue-300 p-3">01</div>
          <div className="text-center bg-blue-300 p-3">02</div>
          <div className="text-center bg-blue-300 p-3">03</div>
          <div className="text-center bg-blue-300 p-3">04</div>
          <div className="text-center bg-blue-300 p-3">05</div>
          <div className="text-center bg-blue-300 p-3">06</div>
          <div className="text-center bg-blue-300 p-3">07</div>
          <div className="text-center bg-blue-300 p-3">08</div>
          <div className="text-center bg-blue-300 p-3">09</div>
        </div>
      </div>
      <div>
        <h3 className="mt-6 text-3xl font-bold">3 Columns Grid</h3>
        <div className="grid grid-cols-3 gap-4">
          <div className="text-center bg-blue-300 p-3">01</div>
          <div className="text-center bg-blue-300 p-3">02</div>
          <div className="text-center bg-blue-300 p-3">03</div>
          <div className="col-span-2 text-center bg-blue-300 p-3">04</div>
          <div className="text-center bg-blue-300 p-3">05</div>
          <div className="text-center bg-blue-300 p-3">06</div>
          <div className="col-span-2 text-center bg-blue-300 p-3">07</div>
        </div>
      </div>
    </div>
  );
}
```

Import the component above into the ***TailwindLab*** page and confirm it renders as shown below.



Figure 2.3.6a - CSS Grid Layout

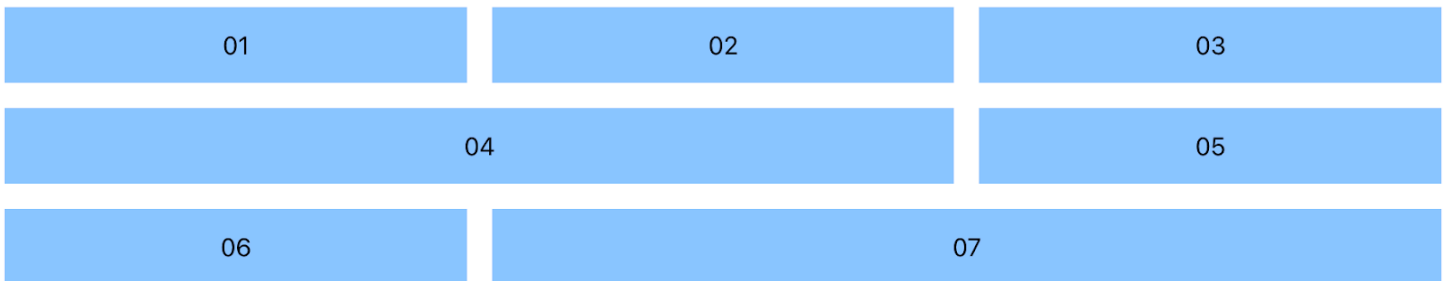


Figure 2.3.6b - CSS Grid Layout

2.4.3 Laying Out HTML Content with Tailwind CSS

It's easy to break a page vertically in HTML with the **p** and **div** tags. It's a little harder to layout content horizontally. Some resort to HTML tables, but this is generally considered a bad practice. Instead, we can use CSS Flexbox or Grid. Tailwind CSS provides powerful utility classes to achieve this functionality efficiently. To practice with Tailwind layouts, append the following to the **TailwindGrids** component using the code below.

app/Labs/Lab2/tailwind/TailwindGrids.tsx

```
<div id="wd-tailwind-grid-system">
  <h2>Grid system</h2>
  <div className="grid grid-cols-2 gap-2">
    <div className="bg-red-500 text-white">
      <h3>Left half</h3>
    </div>
    <div className="bg-blue-500 text-white">
      <h3>Right half</h3>
    </div>
  </div>
  <div className="grid grid-cols-12 gap-2">
    <div className="col-span-4 bg-yellow-500">
      <h3>One third</h3>
    </div>
    <div className="col-span-8 bg-green-500 text-white">
      <h3>Two thirds</h3>
    </div>
  </div>
  <div className="grid grid-cols-12 gap-2">
    <div className="col-span-2 bg-black text-white">
      <h3>Sidebar</h3>
    </div>
    <div className="col-span-8 bg-gray-500 text-white">
      <h3>Main content</h3>
    </div>
    <div className="col-span-2 bg-blue-400">
      <h3>Sidebar</h3>
    </div>
  </div>
</div>
```

</div>

Figure 2.3.3 - Laying Out HTML Content with Tailwind CSS Grids

Specific surrounding directions **top**, **bottom**, **start**, and **end** sides can be configured separately by using **t**, **b**, **s**, and **e**, in combination with the **margin** and **padding** classes, e.g., **mb** refers to **margin top**, **ms** refers to **margin start**.

2.4.4 Implementing Responsive HTML Content with Tailwind Grids

...create new content here...

2.4.4 Implementing Responsive HTML Content with React Bootstrap Grids

[React Bootstrap Grids](#) can adapt to the size of the screen, that is, they can be responsive. We can achieve this by applying more than one **.col** class which only applies for a given window size. To practice with **Bootstrap responsive** grids, create a **BootstrapGrids** component as shown below. Confirm it looks similar to image shown. Resize the browser and confirm that the grid shows 4 columns, then 2 and then just 1.

app/Labs/Lab2/BootstrapGrids.tsx

```
<div id="wd-bs-responsive-grids">
  <h2>Responsive grid system</h2>
  <Row>
    <Col xs={12} md={6} xl={3} className="bg-warning">
      <h3>Column A</h3>
    </Col>
    <Col xs={12} md={6} xl={3} className="bg-primary text-white">
      <h3>Column B</h3>
    </Col>
    <Col xs={12} md={6} xl={3} className="bg-danger text-white">
      <h3>Column C</h3>
    </Col>
    <Col xs={12} md={6} xl={3} className="bg-success text-white">
      <h3>Column D</h3>
    </Col>
  </Row>
</div>
```

Responsive grid system

Column A Column B Column C Column D

Responsive grid system

Column A Column B
Column C Column D

Responsive grid system

Column A
Column B
Column C
Column D

Figure 2.3.4.a - Wide browser window shows 4 columns

Figure 2.3.4.b - Moderate width browser window shows 2 columns

Figure 2.3.4.c - Thin browser window shows only 1 column

Let's try a more dramatic example by adding more content spread over more columns and rows. Copy the HTML code below to the end of **BootstrapGrids.tsx**, and save. Refresh the browser and confirm that the column layout changes as you resize the browser window.

app/Labs/Lab2/BootstrapGrids.tsx

```
<div id="wd-bs-responsive-dramatic">
  <h2>Responsive grid system</h2>
  <Row>
    <Col xs={12} sm={6} md={4} lg={3} xl={2} xxl={1} className="bg-warning">
      <h4>1</h4></Col>
    <Col xs={12} sm={6} md={4} lg={3} xl={2} xxl={1} className="bg-primary text-white">
      <h4>2</h4></Col>
    <Col xs={12} sm={6} md={4} lg={3} xl={2} xxl={1} className="bg-danger text-white">
      <h4>3</h4></Col>
  </Row>
</div>
```

```

<Col xs={12} sm={6} md={4} lg={3} xl={2} xxl={1} className="bg-success text-white">
  <h4>4</h4></Col>
<Col xs={12} sm={6} md={4} lg={3} xl={2} xxl={1} className="bg-warning">
  <h4>5</h4></Col>
<Col xs={12} sm={6} md={4} lg={3} xl={2} xxl={1} className="bg-primary text-white">
  <h4>6</h4></Col>
<Col xs={12} sm={6} md={4} lg={3} xl={2} xxl={1} className="bg-danger text-white">
  <h4>7</h4></Col>
<Col xs={12} sm={6} md={4} lg={3} xl={2} xxl={1} className="bg-success text-white">
  <h4>8</h4></Col>
<Col xs={12} sm={6} md={4} lg={3} xl={2} xxl={1} className="bg-warning">
  <h4>9</h4></Col>
<Col xs={12} sm={6} md={4} lg={3} xl={2} xxl={1} className="bg-primary text-white">
  <h4>10</h4></Col>
<Col xs={12} sm={6} md={4} lg={3} xl={2} xxl={1} className="bg-danger text-white">
  <h4>11</h4></Col>
<Col xs={12} sm={6} md={4} lg={3} xl={2} xxl={1} className="bg-success text-white">
  <h4>12</h4></Col>
</Row>
</div>

```



Figure 2.3.4.d - Extra extra large (xxl) screen size renders 12 column grid



Figure 2.3.4.e - Extra large (xl) screen size renders 6 column grid

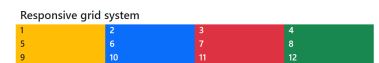


Figure 2.3.4.f - Large (lg) screen size renders 4 column grid

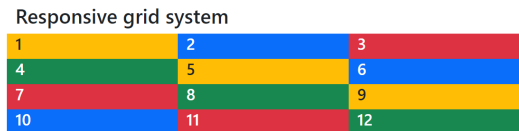


Figure 2.3.4.g - Medium screen (md) size renders 3 column grid

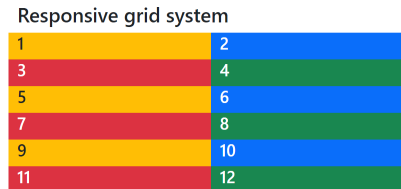


Figure 2.3.4.h - Small (sm) screen size renders 2 column grid

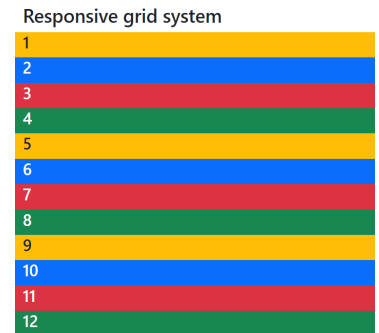


Figure 2.3.4.i - Extra small (xm) screen renders 1 column grid

2.4.5 Hiding and Showing Responsive HTML Content with React Bootstrap

As users shrink or widen the browser window, there may be more or less space to show some content. Bootstrap can configure content to show or hide depending on the width of the screen. As described earlier in [Responsive grids](#), Bootstrap breaks up the screen into 5 sizes: **extra small**, **small**, **medium**, **large**, **extra large**, and **extra extra large**. Create a new component called **ScreenSizeLabel** as shown below which displays and hides different labels at different screen sizes. Add the styling to **index.css** to position the label at the bottom right corner. Create a component as shown below and confirm that the label displays the current screen size when you resize the window.

app/Labs/Lab2/ScreenSizeLabel.tsx

```
export default function ScreenSizeLabel() {
```

app/Labs/Lab2/index.css

```
#wd-screen-size-label {
```

```

return (
  <div id="wd-screen-size-label">
    <div className="d-block d-sm-none">
      XS - Extra Small (<576px) </div>
    <div className="d-none d-sm-block d-md-none">
      S - Small (≥576px) </div>
    <div className="d-none d-md-block d-lg-none">
      M - Medium (≥768px) </div>
    <div className="d-none d-lg-block d-xl-none">
      L - Large (≥992px) </div>
    <div className="d-none d-xl-block d-xxl-none">
      XL - Extra Large (≥1200px) </div>
    <div className="d-none d-xxl-block">
      XXL - Extra Extra Large (≥1400px) </div>
    </div>
  </div>
);

```

```

position: fixed;
bottom: 0;
right: 0;
background-color: black;
color: white;
padding: 5px;
font-size: 12px;
z-index: 1000;
width: 220px;
text-align: center;
}

```

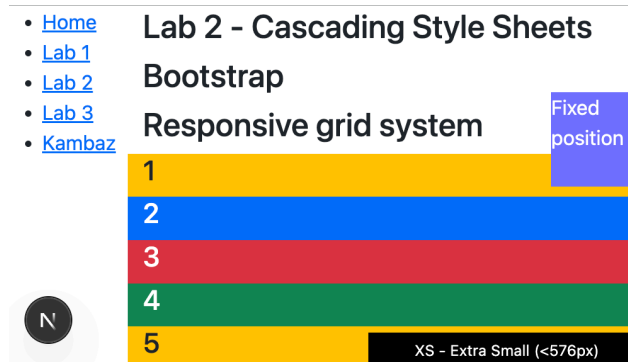


Figure 2.3.5 - Hiding and Showing Responsive Content with React Bootstrap

2.4.6 Styling HTML Tables with React Bootstrap

The [React Bootstrap Table](#) component and classes that enhance the look and feel of common HTML widgets such as tables, lists, and form elements. Let's start with tables. To practice with styling [HTML tables](#), create a component with the code shown below. Refresh the browser and confirm it looks similar to image shown.

app/Labs/Lab2/BootstrapTables.tsx

```

<div id="wd-css-styling-tables">
  <h2>Tables</h2>
  <Table>
    <thead>
      <tr className="table-dark"><th>Quiz</th><th>Topic</th><th>Date</th><th>Grade</th></tr>
    </thead>
    <tbody>
      <tr className="table-warning"><td>Q1</td><td>HTML</td><td>2/3/21</td><td>85</td></tr>
      <tr className="table-danger"><td>Q2</td><td>CSS</td><td>2/10/21</td><td>90</td></tr>
      <tr className="table-primary"><td>Q3</td><td>JavaScript</td><td>2/17/21</td><td>90</td></tr>
    </tbody>
    <tfoot>
      <tr className="table-success"><td colspan={3}>Average</td><td>90</td></tr>
    </tfoot>
  </Table>
</div>

```

Quiz	Topic	Date	Grade
Q1	HTML	2/3/21	85
Q2	CSS	2/10/21	90
Q3	JavaScript	2/17/21	90
Average			90

Figure 2.3.6 - Styling HTML Tables with React Bootstrap

2.4.7 Implementing Responsive Tables with React Bootstrap

In general it is a good practice to minimize the number of scrollbars shown at any one time in a browser screen. In browsers, large amounts of content extends vertically beyond the height of the window, and then scrollbars allow you to access that extra content by scrolling vertically. Sometimes it is necessary to use additional scrollbars when appropriate such as tables or images that might be too wide to fit horizontally. Bootstrap provides tables that can show scrollbars when they don't fit in a screen. To practice with [React Bootstrap Responsive Tables](#), create a component with the code below and confirm it looks similar to image shown. Resize the window and confirm that the table shows a horizontal scroll bar when the screen is too small to fit the table comfortably.

app/Labs/Lab2/BootstrapTables.tsx

```
<div id="wd-css-responsive-tables">
  <h2>Responsive tables</h2>
  <Table responsive>
    <thead>
      <tr><th>Very</th><th>long</th><th>set</th><th>of</th><th>columns</th>
      <th>Very</th><th>long</th><th>set</th><th>of</th><th>columns</th>
      <th>Very</th><th>long</th><th>set</th><th>of</th><th>columns</th>
    </tr>
    </thead>
    <tbody>
      <tr>
        <td>Very</td><td>long</td><td>set</td><td>of</td><td>columns</td>
        <td>Very</td><td>long</td><td>set</td><td>of</td><td>columns</td>
        <td>Very</td><td>long</td><td>set</td><td>of</td><td>columns</td>
      </tr>
    </tbody>
  </Table>
</div>
```

Responsive tables

Very	long	set	of	columns	Very	long	set	of	columns	Very	long	set	of	columns
Very	long	set	of	columns	Very	long	set	of	columns	Very	long	set	of	columns

Figure 2.3.7.a - Responsive tables don't add scrollbars if they fit

Responsive tables

mns	Very	long	set	of	columns	Very	loi
mns	Very	long	set	of	columns	Very	loi

Figure 2.3.7.b - Responsive tables add scrollbars when they don't fit

2.4.8 Styling Lists with React Bootstrap

Another set of React Bootstrap components can make simple HTML lists look more presentable. The [ListGroup and ListGroupItem components](#) can be applied to **ul** and **li** tags correspondingly to make list stand out. The **active** attribute can be applied to an **ListGroupItem** components to highlight it. To practice with **React Bootstrap List Group**, create a component with the code below and confirm it looks similar to image shown in Figure 2.3.8.

app/Labs/Lab2/BootstrapLists.tsx

```
<div id="wd-css-styling-lists">
  <h2>Favorite movies</h2>
  <ListGroup>
    <ListGroupItem active>Aliens</ListGroupItem>
    <ListGroupItem>Terminator</ListGroupItem>
    <ListGroupItem>Blade Runner</ListGroupItem>
    <ListGroupItem>Lord of the Ring</ListGroupItem>
    <ListGroupItem disabled>Star Wars</ListGroupItem>
  </ListGroup>
</div>
```

2.4.9 Styling Hyperlink Lists with React Bootstrap

The same **ListGroup** and **ListGroupItem** Bootstrap components can be configured as **actions** and **href** to implement a list of anchor links. To practice with [React Bootstrap hyperlink lists](#), create a component with the code below and confirm it looks similar to image shown in Figure 2.3.9 and that the links work. Note that you'll need to make this a **client component**.

app/Labs/Lab2/BootstrapLists.tsx

```
<div id="wd-css-hyperlink-list">
  <h3>Favorite books</h3>
  <ListGroup>
    <ListGroupItem action active href="https://en.wikipedia.org/wiki/Dune_(novel)">
      Dune
    </ListGroupItem>
    <ListGroupItem action href="https://en.wikipedia.org/wiki/The_Lord_of_the_Rings">
      Lord of the Rings
    </ListGroupItem>
    <ListGroupItem action href="https://en.wikipedia.org/wiki/The_Forever_War">
      The Forever War
    </ListGroupItem>
    <ListGroupItem action href="https://en.wikipedia.org/wiki/2001:_A_Space_Odyssey_(novel)">
      2001 A Space Odyssey
    </ListGroupItem>
    <ListGroupItem action disabled href="https://en.wikipedia.org/wiki/Ender%27s_Game">
      Ender's Game
    </ListGroupItem>
    <ListGroupItem action onClick={() => alert("New book added")}>
      Add another book
    </ListGroupItem>
  </ListGroup>
</div>
```

Aliens
Terminator
Blade Runner
Lord of the Ring
Star Wars

Dune
Lord of the Rings
The Forever War
2001 A Space Odyssey
Ender's Game
Add another book

Figure 2.3.8 - Favorite movies

Figure 2.3.9 - Favorite books

2.4.10 Styling Forms with React Bootstrap

React Bootstrap has tons of components to style form elements especially to make them friendly for mobile Web applications. To practice with [React Bootstrap form components](#), create a component with the code below and confirm it renders similar to the image shown.

app/Labs/Lab2/BootstrapForms.tsx

```

<div id="wd-css-styling-forms">
  <h2>Forms</h2>
  <FormLabel>Email address</FormLabel>
  <FormControl type="email" placeholder="name@example.com" />
  <FormLabel>Example textarea</FormLabel>
  <FormControl as="textarea" rows={3} />
</div>

```

Email address

Example textarea

Figure 2.3.10 - Styling Forms with React Bootstrap

2.4.11 Styling Dropdowns with React Bootstrap

Dropdowns can also be styled professionally. To practice with [React Bootstrap dropdown styling](#), create a component with the code below and confirm it looks similar to image shown.

app/Labs/Lab2/BootstrapForms.tsx

```

<div id="wd-css-styling-dropdowns">
  <h3>Dropdowns</h3>
  <FormSelect>
    <option value="0" defaultChecked>Open this select menu</option>
    <option value="1">One</option>
    <option value="2">Two</option>
    <option value="3">Three</option>
  </FormSelect>
</div>

```

Open this select menu ▼

✓
Open this select menu

One

Two

Three

Figure 2.3.11 - Styling Dropdowns with React Bootstrap

2.4.12 Styling Switches with React Bootstrap

Checkboxes can be styled to look like switches with React Bootstrap components **FormCheck** with **type="switch"**. To practice with **Bootstrap form switches**, create a component with the code below.

app/Labs/Lab2/BootstrapForms.tsx

```
<div id="wd-css-styling-switches">
  <h3>Switches</h3>
  <FormCheck type="switch" defaultChecked={false} label="Unchecked switch checkbox input"/>
  <FormCheck type="switch" defaultChecked={true} label="Checked switch checkbox input"/>
  <FormCheck type="switch" defaultChecked={false} label="Unchecked disabled switch checkbox input" disabled/>
  <FormCheck type="switch" defaultChecked={true} label="Checked disabled switch checkbox input" disabled/>
</div>
```

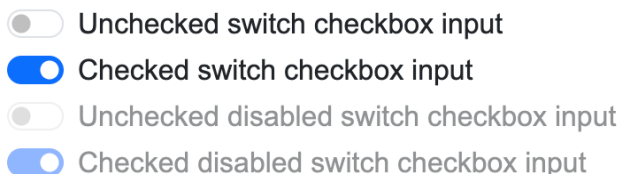


Figure 2.3.12 - Styling Switches with React Bootstrap

2.4.13 Styling Range and Sliders with React Bootstrap

Range input fields render as sliders. To practice with **Bootstrap sliders**, create a component with the code below and confirm it renders similar to image shown.

app/Labs/Lab2/BootstrapForms.tsx

```
<div id="wd-css-styling-range-and-sliders">
  <h3>Range</h3>
  <FormLabel>Example range</FormLabel>
  <FormRange min="0" max="5" step="0.5" />
</div>
```

Example range



Figure 2.3.13 - Styling Range and Sliders with React Bootstrap

2.4.15 Styling Responsive Forms with React Bootstrap

Forms can be configured to display either horizontally or vertically depending on the size of the containing element. To practice with Bootstrap responsive forms, create a component with the code below and confirm it renders similar to image shown. Resize the window to show how the form changes layout as the window resizes.

app/Labs/Lab2/BootstrapForms.tsx

```
<div id="wd-css-responsive-forms-1">
  <h3>Responsive forms</h3>
  <Row className="mb-3" controlId="email1">
    <FormLabel column sm={2}> Email </FormLabel>
    <Col sm={10}>
      <FormControl type="email" defaultValue="email@example.com" />
    </Col>
  </Row>
</div>
```

```

    </Col>
  </Row>
  <Row className="mb-3" controlId="password1">
    <FormLabel column sm={2}> Password </FormLabel>
    <Col sm={10}>
      <FormControl type="password" />
    </Col>
  </Row>
  <Row className="mb-3" controlId="textarea2">
    <FormLabel column sm={2}> Bio </FormLabel>
    <Col sm={10}>
      <FormControl as="textarea" style={{ height: "100px" }} />
    </Col>
  </Row>
</div>

```

Email

Password

Bio

Email

Password

Bio

Figure 2.3.15.a - Styling Responsive Forms with React Bootstrap

Figure 2.3.15.b - Styling Responsive Forms with React Bootstrap

Here's another example. Create a component with the code below and confirm it renders similar to image shown. Resize the window to show how the form changes layout as the window resizes.

app/Labs/Lab2/BootstrapForms.tsx

```

<div id="wd-css-responsive-forms-2">
  <h3>Responsive forms 2</h3>
  <Form>
    <Row className="mb-3" controlId="formHorizontalEmail">
      <FormLabel column sm={2}> Email </FormLabel>
      <Col sm={10}> <FormControl type="email" placeholder="Email" /> </Col>
    </Row>
    <Row className="mb-3" controlId="formHorizontalPassword">
      <FormLabel column sm={2}> Password </FormLabel>
      <Col sm={10}> <FormControl type="password" placeholder="Password" /> </Col>
    </Row>
    <fieldset>
      <Row className="mb-3">
        <FormLabel as="legend" column sm={2}> Radios </FormLabel>
        <Col sm={10}>
          <FormCheck type="radio" label="First radio" name="formHorizontalRadios" defaultChecked />
          <FormCheck type="radio" label="Second radio" name="formHorizontalRadios" />
          <FormCheck type="radio" label="Third radio" name="formHorizontalRadios" />
          <FormCheck type="radio" label="Remember me" name="formHorizontalRadios" />
        </Col>
      </Row>
    </fieldset>
    <Col> <Button type="submit">Sign in</Button> </Col>
  </Form>
</div>

```

Responsive forms

Email

Password

Radios ☒ first radio
☐ second radio
☐ third radio

☐ Remember me

[Sign in](#)

Figure 2.3.15.c - Styling Responsive Forms with React Bootstrap

Responsive forms

Email

Password

Radios ☒ first radio
☐ second radio
☐ third radio

☐ Remember me

[Sign in](#)

Figure 2.3.15.d - Styling Responsive Forms with React Bootstrap

2.4.16 Styling Navigation Tabs with React Bootstrap

Bootstrap provides several common navigation widgets such as tabs, menus, and pills. Let's take a look at tabs first. To practice with Bootstrap tabs, create a component with the code below and confirm it renders similar to image shown.

app/Labs/Lab2/BootstrapNavigation.tsx

```
<div id="wd-css-navigating-with-tabs">
  <h2>Tabs</h2>
  <Nav variant="tabs">
    <NavItem>
      <NavLink href="#/labs/lab2/Active">Active</NavLink>
    </NavItem>
    <NavItem>
      <NavLink href="#/labs/lab2/Link1">Link 1</NavLink>
    </NavItem>
    <NavItem>
      <NavLink href="#/labs/lab2/Link2">Link 2</NavLink>
    </NavItem>
    <NavItem>
      <NavLink href="#/labs/lab2/Disabled" disabled>Disabled</NavLink>
    </NavItem>
  </Nav>
</div>
```



Figure 2.3.16 - Styling Navigation Tabs with React Bootstrap

2.4.17 Styling Navigation Pills with React Bootstrap

Pills are another navigation widget listing several links. Here's an example of using the **Bootstrap Pills** to refactor the **TOC** component with Bootstrap's **pills** variant of the **Nav** component. Confirm the **TOC** highlights the corresponding link as you navigate between the labs.

app/Labs/TOC.tsx

```
import { Nav, NavItem, NavLink } from "react-bootstrap";
import Link from "next/link";
export default function TOC() {
  return (
    <Nav variant="pills">
      <NavItem>
        <NavLink href="/labs" as={Link}>Labs</NavLink>
      </NavItem>
      <NavItem>
        <NavLink href="/labs/lab1" as={Link}>Lab 1</NavLink>
      </NavItem>
      <NavItem>
        <NavLink href="/labs/lab2" as={Link}>Lab 2</NavLink>
      </NavItem>
      <NavItem>
        <NavLink href="/labs/lab3" as={Link}>Lab 3</NavLink>
      </NavItem>
      <NavItem>
        <NavLink href="/" as={Link}>Kambaz</NavLink>
      </NavItem>
      <NavItem>
        <NavLink href="https://github.com/jannunzi">My GitHub</NavLink>
      </NavItem>
    </Nav>
  );
}
```

Labs Lab 1 Lab 2 Lab 3 Kambaz My GitHub

Figure 2.3.17 - Styling Navigation Pills with React Bootstrap

2.4.18 Styling Navigation with React Bootstrap Cards

Cards combine images, titles, paragraphs and buttons into a reusable component that can quickly summarize a topic. To practice with **Bootstrap cards**, create a component with the code below and confirm it renders similar to image shown. Use an image of your own, or [download one from my Flickr account](#) and save it to **public/images/stacked.jpg**. Import all necessary Bootstrap components, e.g, **Card**, and **Button**.

app/Labs/Lab2/BootstrapNavigation.tsx

```
<div id="wd-css-navigating-with-cards">
  <h2> Cards </h2>
  <Card style={{ width: "18rem" }}>
    <CardImg variant="top" src="images/stacked.jpg" />
    <CardBody>
      <CardTitle>Stacking Starship</CardTitle>
      <CardText>
        Stacking the most powerful rocket in history. Mars or bust!
      </CardText>
      <Button variant="primary">Boldly Go</Button>
    </CardBody>
  </Card>
</div>
```

Cards

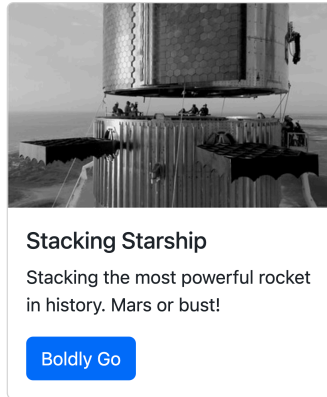


Figure 2.3.18 - Styling Navigation with React Bootstrap Cards

2.5 Styling Kambaz with CSS and Bootstrap

The previous chapter implemented a prototype of the **Kambaz** Web application using plain HTML without overriding the default browser style sheet. This section revisits the **Kambaz** screens using **CSS** and **Bootstrap** to layout and color the screens so they look more like the screenshots provided. Make an effort to style the HTML code to make the screens look as close as possible to their intended look and feel, but don't fret if the screens are not pixel perfect and instead, follow the guidelines and requirements to practice. Several code snippets are provided as a suggestion on how to achieve the layout and style. Feel free to use the code as provided or ignore it to implement as you will. If you do implement the styling on your own, make sure it matches the screenshots provided.

The previous chapter used **table**, **tr**, and **td** elements to layout screens horizontally, but in general this is considered a bad practice and CSS alternatives are preferred. Remove the **table** elements in preparation of using **CSS** instead as shown below.

app/(kambaz)/Layout.tsx

```
<div id="wd-kambaz">
  <table><tbody><tr><td valign="top" width="200">
    <div className="d-flex">
      <div>
        <KambazNavigation />
      </div>
    </td><td valign="top" width="100%">
      <div className="flex-fill">
        {children}
      </div>
    </td></tr></tbody></table>
</div>
```

Do the same for the **Courses** and **Home** screens replacing the **table** elements with **div** elements styled with [Bootstrap flex classes](#) and [display classes](#). Style the **Course** header as shown below and confirm the **CourseNavigation** sidebar and the **Course Status** sidebar appears and hides as you resize the window. Add any missing imports as needed.

app/(kambaz)/courses/[cid]/Layout.tsx

```
<div id="wd-courses">
  <h2 className="text-danger">
    <FaAlignJustify className="me-4 fs-4 mb-1" />
    Course {cid} </h2> <hr />
  <table><tbody><tr><td valign="top" width="200">
```

app/(kambaz)/courses/[cid]/home/page.tsx

```
<div id="wd-home">
  <table><tbody><tr><td valign="top">
    <div className="d-flex" id="wd-home">
      <div className="flex-fill me-3">
        <Modules />
```

```

<div className="d-flex">
  <div className="d-none d-md-block">
    <CourseNavigation />
  </div>
  <div className="flex-fill">
    {children}
  </div>
</div>
</td></tr>
</tbody></table>
</div>

```

```

</div>
</td><td valign="top">
  <div className="d-none d-lg-block">
    <CourseStatus />
  </div>
</td></tr></tbody></table>
</div>

```

2.5.1 Styling the Kambaz Navigation Sidebar

The **Kambaz** Web application has several screens implementing different features. Figure 2.4.1.a bellow illustrates the **Kambaz Navigation** sidebar with the **Dashboard** link selected displaying a grid of courses that allow navigating to different courses. Without CSS the best the previous chapter could do was implement the list of links shown below in Figure 2.4.1.b. With CSS and Bootstrap, this section demonstrates how to style the **Kambaz Navigation** as shown below in Figure 2.4.1.c, as well as configure it to navigate to the **Kambaz Dashboard** and **Courses Home** screens.

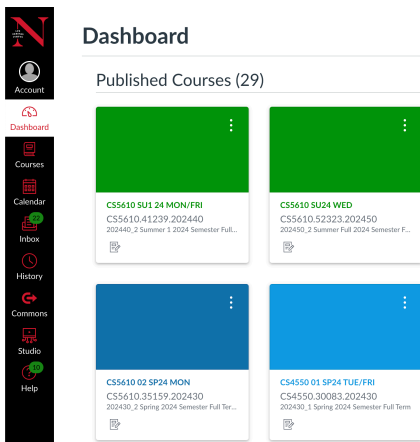


Figure 2.4.1.a - **Kambaz Navigation** and **Dashboard**

[Account](#)
[Dashboard](#)
[Courses](#)
[Calendar](#)
[Inbox](#)
[Labs](#)

Figure 2.4.1.b - **Kambaz Navigation** so far

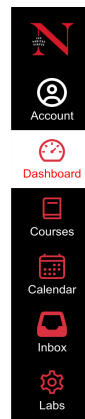


Figure 2.4.1.c - **Kambaz Navigation** this chapter

The previous chapter implemented the **Kambaz Navigation** sidebar in `app/(kambaz)/Navigation.tsx` as rendered above. Using [Bootstrap's Links and Buttons](#) to `Navigation.tsx`, style the sidebar with **CSS** so that it looks more like the sidebar in the screenshot above on the right. Use the code below as a guide to apply the **Bootstrap** classes and **React Icons** to render the **Dashboard** and **Calendar** as icons. Explore other icons for the rest of the links. The icons don't have to match the ones showed here, but should reflect the meaning and intention of the link's text and target screen.

`app/(kambaz)/Navigation.tsx`

```

import { AiOutlineDashboard } from "react-icons/ai";
import { IoCalendarOutline } from "react-icons/io5";
import { LiaBookSolid, LiaCogSolid } from "react-icons/lia";
import { FaInbox, FaRegCircleUser } from "react-icons/fa6";
import { ListGroup, ListGroupItem } from "react-bootstrap";
import Link from "next/link";
export default function KambazNavigation() {
  return (
    <ListGroup className="rounded-0 position-fixed bottom-0 top-0 d-none d-md-block bg-black z-2" style={{ width: 120 }}
      id="wd-kambaz-navigation">
      <ListGroupItem className="bg-black border-0 text-center" as="a"
        target="_blank" href="https://www.northeastern.edu/" id="wd-neu-link">
        
      </ListGroupItem><br />
    </ListGroup>
  )
}

```

```

<ListGroupItem className="border-0 bg-black text-center">
  <Link href="/account" id="wd-account-link" className="text-white text-decoration-none">
    <FaRegCircleUser className="fs-1 text-white" />
    <br />
    Account
  </Link>
</ListGroupItem><br />
<ListGroupItem className="border-0 bg-white text-center">
  <Link href="/dashboard" id="wd-dashboard-link" className="text-danger text-decoration-none">
    <AiOutlineDashboard className="fs-1 text-danger" />
    <br />
    Dashboard
  </Link>
</ListGroupItem><br />
{/* complete styling the rest of the links */}
</ListGroup>
);}

```

The **Northeastern** logo can be found in the [Brand Center website](#). Apply the following **Bootstrap** classes to fix the position of the **Kambaz Navigation** sidebar and stretch it vertically the whole height of the screen.

- **position-fixed** - applies **display: fixed** so that the sidebar stays fixed in one place and doesn't scroll with the rest of the screen
- **bottom-0 top-0** - applies **bottom: 0** and **top: 0** so that the **top** edge is fixed at the **top** of the screen and the **bottom** edge is fixed at the **bottom** of the screen, effectively stretching the height of the element the whole height of the window.
- **d-none d-md-block** - applies **display: none** so that the element is initially hidden. And then applies **display: block** to display the element when the screen width reaches **mid** sized screens, e.g., **768** pixels
- **z-2** - applies **z-index: 2** which brings the element above other elements with a lower **z-index**.
- **bg-black** - applies **background-color: black**

Confirm that the **Kambaz Navigation** sidebar stretches the whole height of the screen, does not scroll with the rest of the **Dashboard** and disappears when the screen is narrow, but appears again when the screen widens.

app/(kambaz)/Navigation.tsx

```

export default function KambazNavigation() {
  return (
    <ListGroup id="wd-kambaz-navigation" style={{ width: 120 }}
      className="rounded-0 position-fixed bottom-0 top-0 d-none d-md-block bg-black z-2">
      ...
    </ListGroup>
  );}

```

The implementation does not need to be pixel perfect, but here are some rough requirements that should be considered when implementing the **Kambaz Navigation**. Use your own access to **Canvas** to help guide your design

- The **Kambaz Account Sign** screen must be the default screen when navigating to **Kambaz**
- Icons must be **red**, except the **Account** icon which must be **white**
- Icons don't have to match exactly, but must be similar. [Use an equivalent React Icons](#)
- The font size and style must be similar
- The whole sidebar must have a black background
- Selected (active) links must have a white background with red text
- Non selected links must have a black background with white text
- The width of the sidebar must be about 110 pixels wide, +/- 5 pixels.
- The icons and text must be centered in the sidebar

Once you are satisfied with the styling of the **Kambaz Navigation** sidebar, visit all the screens and make sure that the **Kambaz Navigation** styling is as shown above. The **Kambaz Navigation** sidebar is now stuck to the left of the screen and

does not scroll with the main content on the right, but the **Dashboard** now renders under the sidebar which overlaps the main content. This is because **position-fixed** removes the **DIV** from the normal layout flow and no longer pushes other content to render below it. This can be fixed by offsetting the main content to the right to compensate for the width of the sidebar. Additionally, the sidebar appears and disappears at different screen sizes, so the offset needs to apply only when the sidebar is present and not apply when the sidebar is hidden. Create class **wd-main-content-offset** in a new **styles.css** file that offsets **Kambaz's** main content by **140** pixels, enough to move the Routes to the right off under the **Kambaz Navigation** sidebar. The class is declared inside a **@media** query that applies the **140** pixel offset only when the screen is wider than **768** pixels, exactly when the sidebar appears. Navigate to the **Dashboard** and confirm that it does not render under the sidebar for all screen sizes

app/(kambaz)/Layout.tsx	app/(kambaz)/styles.css
<pre>import { ReactNode } from "react"; import KambazNavigation from "../Navigation"; import "../styles.css"; export default function KambazLayout(...) { return (<div id="wd-kambaz"> <div className="d-flex"> <div> <KambazNavigation /> </div> <div className="wd-main-content-offset p-3 flex-fill"> {children} </div> </div> </div>); }</pre>	<pre>body { margin: 0; padding: 0; } body * { font-family: Arial, Helvetica, sans-serif; } @media (min-width: 768px) { #wd-kambaz .wd-main-content-offset { padding-left: 140px !important; } }</pre>

2.5.2 Styling the Kambaz Dashboard Screen

The previous chapter implemented the **Kambaz Dashboard** as the default screen when first login into **Kambaz** and it lists courses as shown below. Clicking on a course navigates to that specific course. Since JavaScript has not been officially covered yet, this chapter will just implement a single course and all courses in the **Kambaz Dashboard** will navigate to that same course. Later chapters will implement a data structure representing multiple courses. Courses are rendered in a responsive grid pattern that adapts (responds) to the width of the screen. When the screen is wide, each row of the grid displays several courses, as shown in Figure 2.4.2.a below. As the screen becomes narrower, the courses that don't fit, wrap to the next row as shown in Figures 2.4.2.b, and 2.4.2.c below. When the screen is too narrow, the **Kambaz Navigation** is hidden and courses are displayed in a single column as shown in Figure 2.4.2.d.

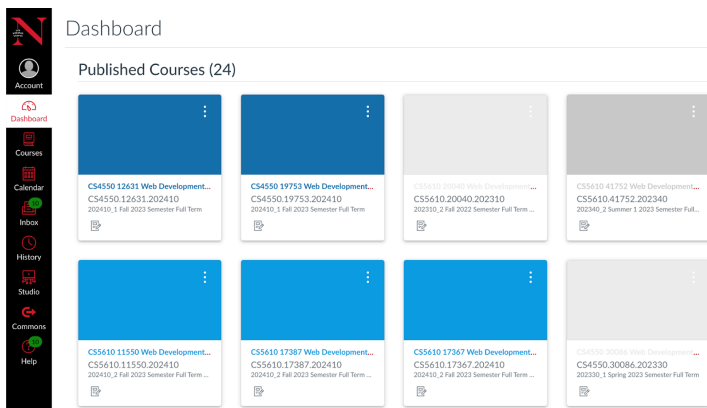


Figure 2.4.2.a - **Dashboard** at widest screen

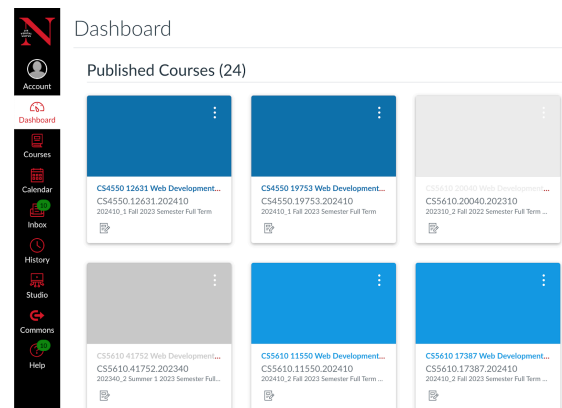


Figure 2.4.2.b - As screen narrows, courses wrap

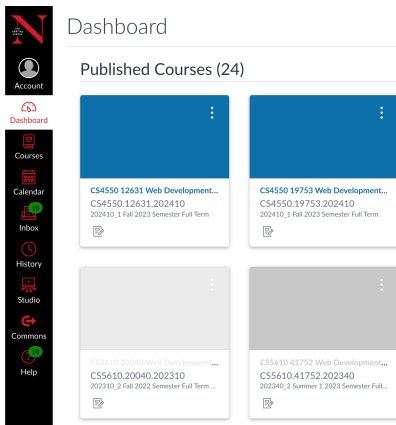


Figure 2.4.2.c - As screen narrows, courses wrap even further

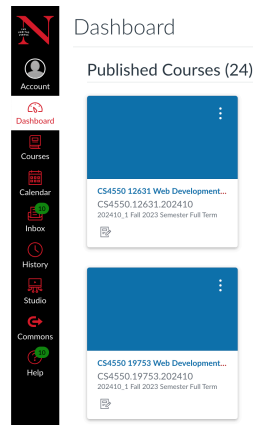


Figure 2.4.2.d - At narrowest screen, just one column of courses shows

The behavior of adapting the layout of the screen and hiding and showing content at various screen sizes is referred to as **Responsive Design**, allowing developers to create content that can be viewed in multiple devices with varying screen sizes. Refactor the current **Dashboard** implementation so that it behaves as described above. Use **React Bootstrap's** responsive [Grid Cards](#) which render a row of [Bootstrap Cards](#) from left to right and wrap as the window becomes narrower. Use the **page.tsx** file shown below as an example. Feel free to come up with your own courses. Replace the course's image `<img>` with **React Bootstrap's** `<CardImg>`, replace the course's title `<h5>` with **React Bootstrap's** `<CardTitle>`, replace the course's description paragraph `<p>` with **React Bootstrap's** `<CardText>`, replace `<button>` with **React Bootstrap's** `<Button>`. Download images for each course and save them to **public/images**. [The code below uses an image of the React logo](#). Feel free to use your own implementation as long as it looks and behaves as described.

app/(kambaz)/dashboard/page.tsx

```
...
<div id="wd-dashboard">
  <h1 id="wd-dashboard-title">Dashboard</h1> <hr />
  <h2 id="wd-dashboard-published">Published Courses (12)</h2> <hr />
  <div id="wd-dashboard-courses">
    <Row xs={1} md={5} className="g-4">
      <Col className="wd-dashboard-course" style={{ width: "300px" }}>
        <Card>
          <Link href="/courses/1234/home"
            className="wd-dashboard-course-link text-decoration-none text-dark">
            <CardImg variant="top" src="/images/reactjs.jpg" width="100%" height={160}/>
            <CardBody>
              <CardTitle className="wd-dashboard-course-title text-nowrap overflow-hidden">CS1234 React JS</CardTitle>
              <CardText className="wd-dashboard-course-description overflow-hidden" style={{ height: "100px" }}>
                Full Stack software developer</CardText>
              <Button variant="primary">Go</Button>
            </CardBody>
          </Link>
        </Card>
      </Col>
      <Col className="wd-dashboard-course" style={{ width: "300px" }}> ... Another course ... </Col>
      <Col className="wd-dashboard-course" style={{ width: "300px" }}> ... Another course ... </Col>
    </Row>
  </div></div>
...
```

Your implementation does not need to pixel perfect, but here are some rough requirements you should consider when implementing the **Kambaz Dashboard**. Use your own access to **Canvas** to help guide your design

- The **Dashboard** link must be rendered as selected in the **Kambaz Navigation** sidebar with red text, red icon, and white background
- A **Dashboard** title must appear at the top of the screen as shown
- Clicking the **Dashboard** link must display the **Kambaz Dashboard** screen

- Clicking any course in **Kambaz Dashboard**, reveals the **Home** screen implemented in previous chapters
- A **horizontal rule** (`<hr/>`) must appear below the title as shown
- A **Published Courses** sub title must appear as shown
- At least 7 courses must be rendered under the **Published Courses** as shown
- Courses must render in a grid of rows and columns as shown
- Use **Grid Cards** and **Bootstrap Cards** must to render each course as a card with an image or solid color at the top and the course's title and description, linked to the course **Home** screen
- The widths of the courses as shown are about 300 pixels, and don't change as the window narrows
- There must be white spacing between courses above and below of between 30 and 40 pixels
- There must be white spacing between the right most edge of the **Kambaz Navigation** sidebar and the left most edge of the left most course
- When the window is at its widest, rows should fit at least 4 courses per row
- When the window shrinks and courses don't fit, then courses should wrap to the next row

2.5.3 Styling the Course Navigation Sidebar

Clicking a course in the **Kambaz Dashboard** navigates to that course's **Home** screen. For now, implement the **Kambaz Dashboard** to navigate to the same course. Later chapters will introduce **JavaScript** and revisit this implementation to navigate to the corresponding course. The previous chapter implemented the **Course Navigation** sidebar in `app/(kambaz)/courses/[cid]/Navigation.tsx`. Style the **Course Navigation** sidebar to look more like the one from **Canvas**. The code below illustrates how to apply styles to the **Courses Navigation** sidebar so that it renders closer to the target look and feel shown here on the right. Refactor the list of links using **Bootstrap List and buttons** so that the clickable area is wider and easier to use in smaller screens. The `styles.css` file is implemented at the root of the **Kambaz** folder and is the same we used in the **Kambaz Navigation** sidebar and is documented earlier in this document. Feel free to use and modify the code below as necessary or implement your own version as long as it looks and behaves as shown.

`app/(kambaz)/courses/[cid]/Navigation.tsx`

```
import Link from "next/link";
export default function CourseNavigation() {
  return (
    <div id="wd-courses-navigation" className="wd list-group fs-5 rounded-0">
      <Link href="/courses/1234/home" id="wd-course-home-link"
        className="list-group-item active border-0"> Home </Link><br />
      <Link href="/courses/1234/modules" id="wd-course-modules-link"
        className="list-group-item text-danger border-0"> Modules </Link><br />
      <Link href="/courses/1234/piazza" id="wd-course-piazza-link"
        className="list-group-item text-danger border-0"> Piazza </Link><br />
      <Link href="/courses/1234/zoom" id="wd-course-zoom-link"
        className="list-group-item text-danger border-0"> Zoom </Link><br />
      <Link href="/courses/1234/assignments" id="wd-course-assignments-link"
        className="list-group-item text-danger border-0"> Assignments </Link><br />
      <Link href="/courses/1234/quizzes" id="wd-course-quizzes-link"
        className="list-group-item text-danger border-0"> Quizzes </Link><br />
      <Link href="/courses/1234/people/Table" id="wd-course-people-link"
        className="list-group-item text-danger border-0"> People </Link><br />
    </div>
  );
}
```

The CSS rule in `styles.css` shown below overrides **Bootstrap** classes `list-group`, `list-group-item`, and `active` so that the sidebar renders with a white background color, red text, and black border of the highlighted link.

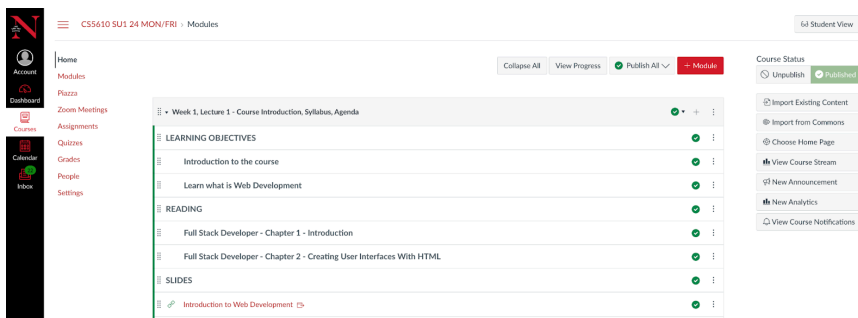
`app/(kambaz)/styles.css`

```
...
.list-group.wd > .list-group-item.active {
  color: black;
```

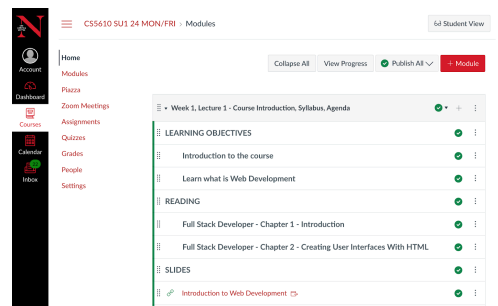
```
background-color: white;
border-left: 3px solid black !important;
}
```

2.5.4 Styling the Modules Screen

The **Modules** in the **Modules Screen** is also used in **Home Screen** so style the **Modules** first. Clicking on a course in the **Kambaz Dashboard** displays the course's **Home** screen as shown below. The **Course Navigation** sidebar provides various links to navigate to different screens related to the course such as **Home**, **Modules**, **Assignments**, and **Grades**. The **Course Home** screen is displayed by default and its link is rendered as selected when you first navigate to a course from the **Dashboard**. In a wide screen the **Course Home** screen renders 4 columns as shown in the Figure 2.4.4.a shown below. The first column is the **Kambaz Navigation** sidebar, the second column is the **Course Navigation** sidebar, and the third column contains the **Course Home** showing a list of modules starting at **Week 1**. The last column are various buttons and links we'll refer to as the **Course Status**. If the window narrows below some threshold, the last column is hidden and the third column takes the remaining width as shown in Figure 2.4.4.b below.



2.4.4.a. Course Home screen renders 4 columns in a wide screen



2.4.4.b. Course Status hides as screen narrows

If the window narrows below some other threshold, the first and second column are hidden, and the third column takes the remaining width as shown in Figure 2.4.4.c shown below. Even though the **Kambaz Navigation** is hidden, it is still available by clicking the top left icon and displays as shown in screenshot Figure 2.4.4.d shown below. The **Course Navigation** is also hidden, but available by clicking the top right corner icon and displays as shown in Figure 2.4.4.e shown below.

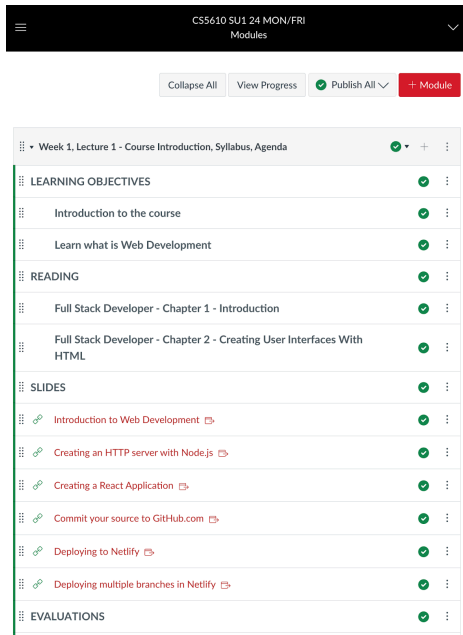


Figure 2.4.4.c - Hide **Kambaz** and **Courses Navigation** at narrowest screen

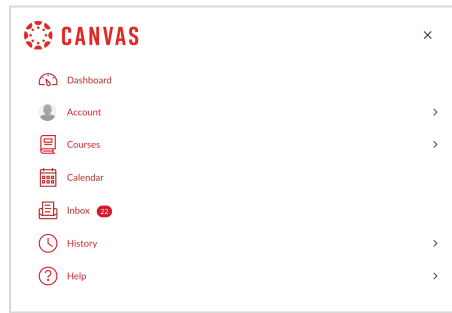


Figure 2.4.4.d - Click top left icon to show **Kambaz Navigation**

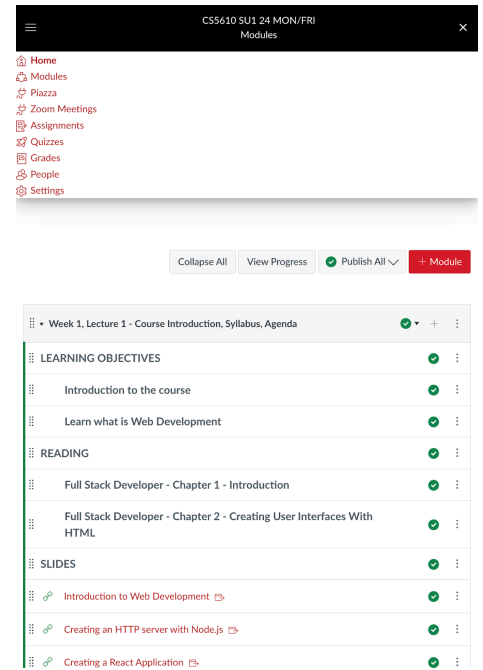
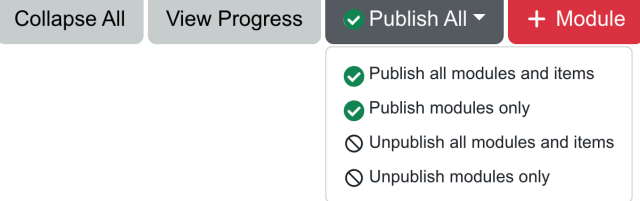


Figure 2.4.4.e - Click top right icon to show **Courses Navigation**

Since **Course Home** screen's main content is the **Modules**, style those first as shown in Figures 2.4.4 below. At the top of the **Modules** there's a collection of buttons and dropdowns including **Collapse All**, **View Progress**, **Publish All**, **Module** as shown on the right. First, create the green circular checkmark in a **GreenCheckmark** component as shown below.



`app/(kambaz)/courses/[cid]/modules/GreenCheckmark.tsx`

```
import { FaCheckCircle, FaCircle } from "react-icons/fa";
export default function GreenCheckmark() {
  return (
    <span className="me-1 position-relative">
      <FaCheckCircle style={{ top: "2px" }} className="text-success me-1 position-absolute fs-5" />
      <FaCircle className="text-white me-1 fs-6" />
    </span>);}
```



Style the **Modules Screen** screen and **Home Screen** to look and behave as shown in the screenshots below.

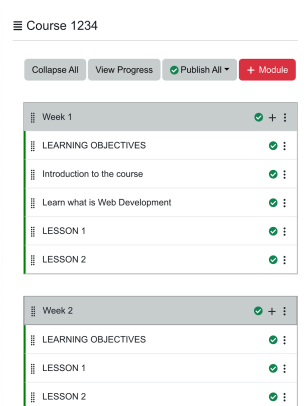
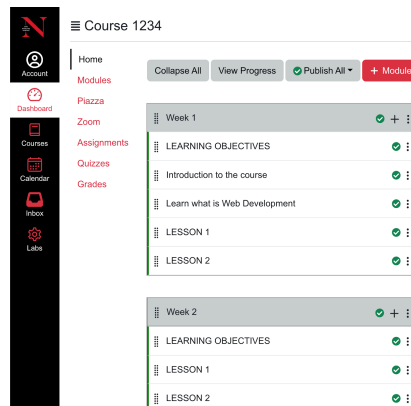
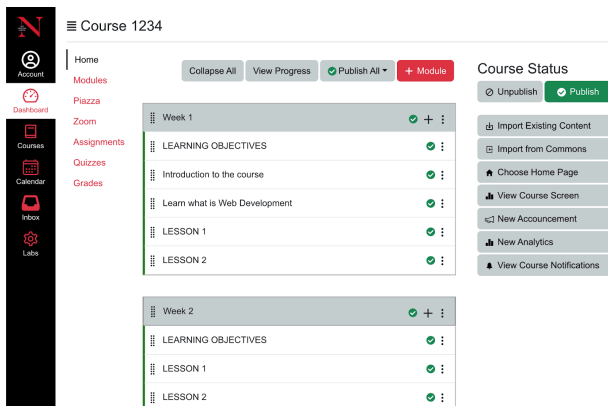


Figure 2.4.4.f - **Course Home** screen renders 4 columns in a wide screen

Figure 2.4.4.g - **Course Status** hides as screen narrows

Figure 2.4.4.h - Hide **Kambaz** and **Courses Navigation** at narrowest screen

Implement the controls at the top of the **Module** in a new **ModulesControls** component. Feel free to use the code below as an example or implement your own as long as it looks and behaves as described.

`app/(kambaz)/courses/[cid]/modules/modulesControls.tsx`

```
import { Button, Dropdown, DropdownItem, DropdownMenu, DropdownToggle } from "react-bootstrap";
import { FaPlus } from "react-icons/fa6";
import GreenCheckmark from "../GreenCheckmark";
export default function ModulesControls() {
  return (
    <div id="wd-modules-controls" className="text-nowrap">
      <Button variant="danger" size="lg" className="me-1 float-end" id="wd-add-module-btn">
        <FaPlus className="position-relative me-2" style={{ bottom: "1px" }} />
        Module
      </Button>
      <Dropdown className="float-end me-2">
        <DropdownToggle variant="secondary" size="lg" id="wd-publish-all-btn">
          <GreenCheckmark /> Publish All
        </DropdownToggle>
        <DropdownMenu>
          <DropdownItem id="wd-publish-all">
            <GreenCheckmark /> Publish All
          </DropdownItem>
          <DropdownItem id="wd-publish-all-modules-and-items">
            <GreenCheckmark /> Publish all modules and items
          </DropdownItem>
          <DropdownItem id="wd-publish-modules-only">
            <GreenCheckmark /> Publish modules only
          </DropdownItem>
          { /* Create two more items with IDs wd-unpublish-all-modules-and-items and wd-unpublish-modules-only with
            labels Unpublish all modules and items and Unpublish modules only */ }
        </DropdownMenu>
      </Dropdown>
      { /* Implement the View Progress and Collapse All buttons with IDs wd-view-progress and wd-collapse-all */ }
    </div>
  );
}
```

Import the component at the top of the **Modules** components and confirm they look as shown in Figures 2.4.4.f through 2.4.4.h. You might need to add several break elements to add some vertical spacing. Refactor the **Modules** component by replacing HTML tags **ul** and **li** with **Bootstrap** components **ListGroup** and **ListGroupItem** as shown below. The following code illustrates how you might implement and style modules **Week 1** and **Week 2**. Feel free to use the code as a guide, or implement your own version as long as it looks and behaves as described. Override the **btn-secondary** and **bg-secondary** **Bootstrap** classes and implement a new border class as show in the **style.css** below.

`app/(kambaz)/courses/[cid]/modules/page.tsx`

```
<div>
  <ModulesControls /><br /><br /><br /><br />
  <ListGroup className="rounded-0" id="wd-modules">
    <ListGroupItem className="wd-module p-0 mb-5 fs-5 border-gray">
      <div className="wd-title p-3 ps-2 bg-secondary"> Week 1 </div>
      <ListGroup className="wd-lessons rounded-0">
        <ListGroupItem className="wd-lesson p-3 ps-1">
          LEARNING OBJECTIVES </ListGroupItem>
        <ListGroupItem className="wd-lesson p-3 ps-1">
          Introduction to the course </ListGroupItem>
        <ListGroupItem className="wd-lesson p-3 ps-1">
          Learn what is Web Development </ListGroupItem>
      </ListGroup>
    </ListGroupItem>
    <ListGroupItem className="wd-module p-0 mb-5 fs-5 border-gray">
      <div className="wd-title p-3 ps-2 bg-secondary"> Week 2 </div>
      <ListGroup className="wd-lessons rounded-0">
```

`app/(kambaz)/styles.css`

```
body { margin: 0; padding: 0; }
body * { font-family:
  Arial, Helvetica,
  sans-serif; }
@media (min-width: 768px) {
  #wd-kambaz .wd-main-content-offset {
    padding-left: 140px !important;
  }
}

.btn-secondary {
  --bs-btn-color: #000 !important;
  --bs-btn-bg: #c7cdd1 !important;
  --bs-btn-border-color: #c7cdd1 !important;
  --bs-btn-hover-color: #fff !important;
}

.bg-secondary {
```

```

<ListGroupItem className="wd-lesson p-3 ps-1">
  LESSON 1 </ListGroupItem>
<ListGroupItem className="wd-lesson p-3 ps-1">
  LESSON 2 </ListGroupItem>
</ListGroup>
</ListGroupItem>
</ListGroup>
</div>

```

```

--bs-secondary-rgb: 199, 205, 209;
}
.list-group-item {
  --bs-list-group-border-color: gray;
}
.wd-lesson {
  border-left: 3px solid green !important;
}

```

Each module title and lesson has a set of control buttons at the right end of the title as shown here on the right. Implement each set of control buttons as separate component. Start with the **LessonControlButtons** with the **GreenCheckmark** plus a vertical ellipsis as shown below. Using the component below as an example, implement **ModuleControlButtons** with an additional **BsPlus** icon.



`app/(kambaz)/courses/[cid]/modules/LessonControlButtons.tsx`

```

import { IoEllipsisVertical } from "react-icons/io5";
import GreenCheckmark from "../GreenCheckmark";
export default function LessonControlButtons() {
  return (
    <div className="float-end">
      <GreenCheckmark />
      <IoEllipsisVertical className="fs-4" />
    </div> );}

```



Add the new **LessonControlButtons** and **ModuleControlButtons** components to the **Modules** component as shown below.

`app/(kambaz)/courses/[cid]/modules/page.tsx`

```

<ModulesControls /><br /><br />
<ListGroup className="rounded-0" id="wd-modules">
  <ListGroupItem className="wd-module p-0 mb-5 fs-5 border-gray">
    <div className="wd-title p-3 ps-2 bg-secondary">
      <BsGripVertical className="me-2 fs-3" /> Week 1 <ModuleControlButtons />
    </div>
  <ListGroup className="wd-lessons rounded-0">
    <ListGroupItem className="wd-lesson p-3 ps-1">
      <BsGripVertical className="me-2 fs-3" /> LEARNING OBJECTIVES <LessonControlButtons />
    </ListGroupItem>
    <ListGroupItem className="wd-lesson p-3 ps-1">
      <BsGripVertical className="me-2 fs-3" /> Introduction to the course <LessonControlButtons />
    </ListGroupItem>
    ...
  </ListGroup>
</ListGroupItem>
</ListGroup>

```

2.5.5 Styling the Home Screen

Since the **Modules** in the **Modules Screen** are also used in the **Home Screen**, we almost done with the **Home Screen**. The only thing left to style is the the **CourseStatus** component at the right of the **Home Screen** contains several buttons. Here's an example how the first few buttons can styled. Implement and style the rest of the buttons so that the **Home Screen** looks like and behaves as documented in the screen shot in Figures 4.4. Your implementation does not need to pixel perfect. Use your own access to **Canvas** to help guide your design.

`app/(kambaz)/courses/[cid]/home/Status.tsx`

```
import { MdDoNotDisturbAlt } from "react-icons/md";
import { FaCheckCircle } from "react-icons/fa";
import { BiImport } from "react-icons/bi";
import { LiaFileImportSolid } from "react-icons/lia";
import { Button } from "react-bootstrap";
{/* Find more icons */}
export default function CourseStatus() {
  return (
    <div id="wd-course-status" style={{ width: "350px" }}>
      <h2>Course Status</h2>
      <div className="d-flex">
        <div className="w-50 pe-1">
          <Button variant="secondary" size="lg" className="w-100 text-nowrap">
            <MdDoNotDisturbAlt className="me-2 fs-5" /> Unpublish </Button> </div>
          <div className="w-50">
            <Button variant="success" size="lg" className="w-100">
              <FaCheckCircle className="me-2 fs-5" /> Publish </Button> </div>
        </div>
        <br />
        <Button variant="secondary" size="lg" className="w-100 mt-1 text-start">
          <BiImport className="me-2 fs-5" /> Import Existing Content </Button>
        <Button variant="secondary" size="lg" className="w-100 mt-1 text-start">
          <LiaFileImportSolid className="me-2 fs-5" /> Import from Commons </Button>
        </div>
      </div>
    </div>
  );
}
```

2.5.6 Implementing the People Screen

The **People** screen displays a list of students, teaching assistants, and faculty associated with the selected course. This section implements and styles a prototype of the screen.

Home	Name	Login ID	Section	Role	Last Activity	Total Activity
Modules	 Tony Stark	001234561S	S101	STUDENT	2020-10-01T00:00:00.000Z	10:21:32
Piazza	 Bruce Wayne	001234562S	S101	STUDENT	2020-11-02T00:00:00.000Z	15:32:43
Zoom	 Steve Rogers	001234563S	S101	STUDENT	2020-10-02T00:00:00.000Z	23:32:43
Assignments	 Natasha Romanoff	001234564S	S101	TA	2020-11-05T00:00:00.000Z	13:23:34
Quizzes	 Thor Odinson	001234565S	S101	STUDENT	2020-12-01T00:00:00.000Z	11:22:33
Grades	 Bruce Banner	001234566S	S101	STUDENT	2020-12-01T00:00:00.000Z	22:33:44
People						

Figure 2.4.6 - The People Screen displaying a table of people enrolled in the course

The component below implements the **People** screen as a table that displays each user as a separate row. Practice by showing a total of three users. Navigate to a course from the **Dashboard** and then to the **People** screen to confirm that it renders as shown above.

`app/(kambaz)/courses/[cid]/people/table/page.tsx`

```
import { Table } from "react-bootstrap";
import { FaUserCircle } from "react-icons/fa";
export default function PeopleTable() {
  return (
    <div id="wd-people-table">
      <Table striped>
        <thead>
```

```

<tr><th>Name</th><th>Login ID</th><th>Section</th><th>Role</th><th>Last Activity</th><th>Total Activity</th></tr>
</thead>
<tbody>
<tr><td className="wd-full-name text-nowrap">
  <FaUserCircle className="me-2 fs-1 text-secondary" />
  <span className="wd-first-name">Tony</span>{" "}
  <span className="wd-last-name">Stark</span></td>
<td className="wd-login-id">001234561S</td>
<td className="wd-section">S101</td>
<td className="wd-role">STUDENT</td>
<td className="wd-last-activity">2020-10-01</td>
<td className="wd-total-activity">10:21:32</td></tr>
  { /* Add at least 3 more users such as Bruce Wayne, Steve Rogers, and Natasha Romanoff */ }
</tbody>
</Table>
</div> );}

```

2.5.7 Styling the Assignments Screen (On Your Own)

Clicking **Assignments** in the **Course Navigation** sidebar displays the **Assignments** screen as shown on the right.

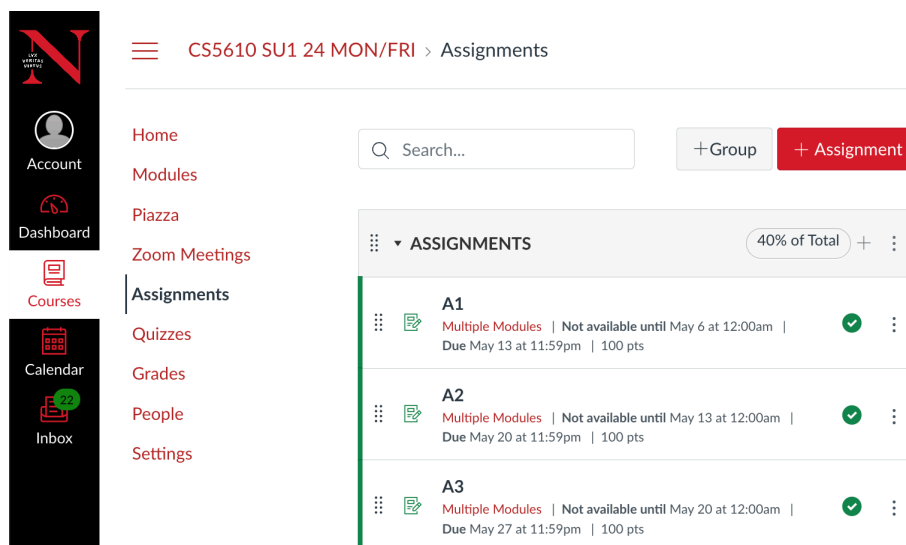


Figure 2.4.7 - The Assignments Screen Screenshot

On your own, use the implementation of the **Home** and **Modules Screens** as examples, and create and style the **Assignments Screen** as shown here on the right. The implementation does not have to be pixel perfect, but make an effort to use icons, colors, spacing, font size, as close as possible to the screen shot shown here on the right. Make an effort to style the **Assignments Screen** as shown above, but it doesn't need to be pixel perfect. Here are some requirements you need to follow

- Buttons must be floated to the right as shown
- The **Search for Assignment** text field must render as shown including the **placeholder** text, the **magnifying glass**, and justified to the left.
- Use **React Icons** similar to the ones shown
- The **Group** and **Assignment** buttons must be justified to the right, in the color shown, and **plus** icons. Use the same colors used in the **Modules** and **Home Screen**
- White spaces around and between content must be rendered with **Bootstrap's** margin and padding classes
- The border to the left of the line items must be rendered green as shown
- The titles of the assignments, **A1**, **A2**, etc, must be rendered as shown
- The subtext under the titles such as the due dates, start times, and points, must render as shown, but the dates and times can be different

2.5.8 Styling Edit Assignment Screen (On Your Own)

Clicking on the title of any assignment displays the **Edit Assignment Screen** as shown below in Figure 2.4.8. For now this screen displays the same content regardless which assignment you click. In later chapters the content will be different depending which assignment you click. The screen provides a form elements for faculty to edit the assignment including the **Assignment Name**, **Description**, **Points**, and **Due Date**. Use **React Bootstrap Components** such as **Form**, **Form.Group**, and **FormLabel**, to style the **Edit Assignment** screen as close to the screen shot here on the right. On your own, make an effort to style the screen as close as possible to the wireframes provided, but it is not required to be pixel perfect.

CS5610 SU1 24 MO... > Assignments > A1

202440_2 Summer 1 2024 S...

Assignment Name

A1

The assignment is **available online**

Submit a link to the landing page of your Web application running on [Netlify](#).

The landing page should include the following:

- Your full name and section
- Links to each of the lab assignments
- Link to the [Kanbas](#) application
- Links to all relevant source code repositories

The [Kanbas](#) application should include a link to navigate back to the landing page.

Points 100

Assignment Group ASSIGNMENTS

Display Grade as Percentage

Submission Type

Online

Online Entry Options

☐ Text Entry

☒ Website URL

☐ Media Recordings

☐ Student Annotation

☐ File Uploads

Assign to

Everyone

Due

May 13, 2024, 11:59 PM

Available from

May 6, 2024, 12:00 AM

Until

Cancel Save

Figure 2.4.8 - The Assignment Editor Screen Screenshot

2.5.9 Styling the Account Screens (On Your Own)

Using Bootstrap classes and the styling of the **Course Navigation** sidebar as an example, style the **Account Screens** so that they look as shown in Figures 2.4.9. Using **form** and **button** Bootstrap classes, style the **Sign In** screen as shown below. Also style the **Account Navigation** sidebar based on the **Course Navigation** sidebar.

`app/(kambaz)/account/signin/page.tsx`

```
import Link from "next/link";
export default function Signin() {
  return (
    <div id="wd-signin-screen">
      <h1>Sign in</h1>
      <FormControl id="wd-username"
        placeholder="username"
        className="mb-2"/><br />
      <FormControl id="wd-password"
        placeholder="password" type="password"
        className="mb-2"/><br />
      <Link id="wd-signin-btn"
        href="/account/profile"
        className="btn btn-primary w-100 mb-2">
        Sign in </Link><br />
      <Link id="wd-signup-link" href="/account/signup">Sign up</Link>
    </div> );}
```

Using the **Sign In** screen as an example, style the **Sign Up** and **Profile** screens so that they look as shown below.

Signin

Signin
[Signup](#)

Signup

Signup
[Signin](#)

Profile

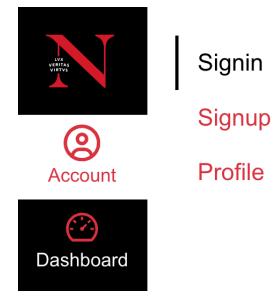
Signout

Figure 2.4.9.a
Styled Signin Screen

Figure 2.4.9.b
Styled Signup Screen

Figure 2.4.9.c
Styled Profile Screen

Figure 2.4.9.d
Styled Account Navigation

2.6 Delivery

1. In the same React application created in earlier chapters, **kambaz-next-js**, complete all the exercises described in this document
2. In a branch called **a2**, add, commit and push the source code of the React application **kambaz-next-js** to the same remote source repository in **GitHub.com** created in an earlier chapter. Here's an example of how to add, commit and push your code

```
$ git checkout -b a2
$ git add .
```

```
$ git commit -am "a2 CSS and Bootstrap"
$ git push
```

3. Deploy the **a2** branch to the same **Vercel** created in an earlier chapter. Configure Vercel to deploy all branches to separate URLs. From your Vercel's dashboard go to **Site settings > Build & deploy > Branches > Branch deploys** and select **All**. Now each time you commit to a branch, the application will be available at a URL that contains the name of the branch
4. Push the source code of the React application **kambaz-next-js** to a remote source repository in **GitHub.com** as described
5. **labs/page.tsx** contains a **TOC.tsx** that references each of the labs and Kambaz. Add a link to your repository in GitHub. The link should have an **ID** attribute with a value of **wd-github**.
6. In **labs/page.tsx**, add your full name: first name first and last name second. Use the same name as in Canvas.
7. Deploy the branch for this chapter to your **kambaz-next-js** React application to **Vercel** as described.
8. As a deliverable in **Canvas**, submit the URL to the **a2** branch deployment of your React application running on Vercel.